Wear-Aware Routing Algorithm for Network-on-Chip Interconnects Using Linear Approximate Reinforcement Learning

by

Farhan Ishraq

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR'S OF APPLIED SCIENCE

in

The Faculty of Applied Science

(Engineering Physics)

THE UNIVERSITY OF BRITISH COLUMBIA

(Vancouver)

April 2026

© Farhan Ishraq 2026

The following individuals certify that they have read, and recommend to the Faculty of Applied Science for acceptance, the thesis entitled:

Wear-Aware Routing Algorithm for Network-on-Chip Interconnects Using Linear Approximate Reinforcement Learning

submitted by **Farhan Ishraq** in partial fulfillment of the requirements for the degree of **Bachelor of Applied Science** in **Engineering Physics**.

Examining Committee:

Andre Ivanov, Professor, Electrical and Computer Engineering, UBC
Supervisor

Abstract

As chip multiprocessors scale, the reliability of Network-on-Chip (NoC) interconnects becomes increasingly critical due to aging mechanisms such as electromigration (EM) and hot-carrier injection (HCI), which are strongly influenced by traffic-induced switching activity and temperature. Conventional routing algorithms, such as dimension-order routing (DOR), are oblivious to these effects and can lead to uneven stress distribution and premature failure of network components. This work presents a reinforcement learning (RL)-based adaptive routing framework that incorporates aging awareness directly into routing decisions. Each router operates as a distributed RL agent using linear function approximation, where EM and HCI based mean time to failure (MTTF) estimates are integrated into the feature representation and reward function. The proposed approach is evaluated using the gem5 Garnet cycle-accurate simulator on an 8×8 mesh across multiple synthetic traffic patterns, and is compared against both DOR and the state-of-the-art aging-aware Clotho routing scheme. Results show that the RL-based approach achieves up to $\sim 5\times$ improvement in EM lifetime and consistent gains in HCI lifetime compared to DOR. Compared to Clotho, the proposed method achieves improved reliability under several structured traffic patterns, while maintaining competitive packet latency across all evaluated scenarios. This work highlights the potential of reinforcement learning as a scalable and flexible framework for reliability-aware packet routing in NoC.

Lay Summary

Modern computer chips rely on internal networks to move data between many processing units, but these connections can wear out over time, especially when some paths are used more than others. Traditional routing methods send data along fixed paths, which can overload certain parts of the network and shorten the chip's lifespan. This thesis explores a learning-based approach where the network adapts its routing decisions based on current conditions, spreading traffic more evenly to reduce stress on heavily used components. By combining this approach with models of how hardware wears out, the system learns to avoid paths that are likely to fail sooner. Our results show that this method can significantly extend the lifetime of the network while maintaining or even improving communication speed, highlighting a promising direction for building more reliable and efficient computer systems.

Preface

The study is based on work conducted at the Ivanov SoC Lab at UBC under the guidance of Professor Andre Ivanov. I was responsible for designing and running experiments, reviewing and presenting relevant literature, and compiling the results. Much consultation was taken with Ian Hill, and Roozmehr Jalilian whose suggestions, insights, and expertise helped guide the project.

Table of Contents

Abstract iii

Lay Summary iv

Preface v

Table of Contents vi

List of Figures viii

List of Abbreviations ix

Acknowledgements x

1 Introduction 1
- 1.1 Problem Statement 4
- 1.2 Scope of Research 5

2 Literature Review 7
- 2.1 Introduction to Routing Algorithms in NoCs 7
- 2.2 Congestion-Aware Routing Algorithms 8
- 2.3 Fault-Tolerant Routing Algorithms 10
- 2.4 Aging-Aware Routing Algorithms 12
- 2.5 Motivation for Learning-Based Aging-Aware Routing 14

3 Aging-Aware Routing Algorithm System Design 16
- 3.1 Process Variation 16
- 3.2 Dynamic Temperature Variation 20
- 3.3 Aging Models 21
- 3.4 Reinforcement Learning-Based Routing Policy 25
- 3.5 Deadlock Avoidance 30

4 Evaluation 33
4.1 Simulation Setup 33
4.2 Garnet Synthetic Traffic and Evaluation Metrics 34
4.3 Results 35

5 Limitations and Future Work 43
5.1 Limitations 43
5.2 Future Work 43

6 Conclusion 45

Bibliography 46

Appendices

A Mean Acceleration Factor 53
A.1 Mean Acceleration Factor (AF) Trends Across Injection Rates 53

B Access to Research Work 56
B.1 Code 56

List of Figures

1.1 Communication Architecture 2
1.2 CMP Failure Modes 3
1.3 Routing Algorithms 4

3.1 Per-router process variation maps 18
3.2 Per-router temperature map 21
3.3 Reinforcement Learning Framework 26
3.4 NoC Router Microarchitecture 30
3.5 Deadlock Formation 31

4.1 Mean EM Wear in Links 35
4.2 Mean HCI Wear in Routers 36
4.3 Average Packet Latency (Transpose, Bit Reverse, Bit Rotation) 39
4.4 Average Packet Latency under Bit Rotation Traffic. 40
4.5 Average Packet Latency (Uniform, Shuffle) 41
4.6 Average Packet Latency (Tornado, Neighbor) 42

A.1 Uniform Random Traffic. 53
A.2 Shuffle Traffic. 53
A.3 Transpose Traffic. 54
A.4 Bit Reverse Traffic. 54
A.5 Bit Rotation Traffic. 54
A.6 Neighbor Traffic. 55
A.7 Tornado Traffic. 55

List of Abbreviations

AF	Acceleration Factor
BTI	Bias Temperature Instability
CMP	Chip Multiprocessor
DBAR	Destination-Based Adaptive Routing
DOR	Dimension-Order Routing
DP	Dynamic Programming
DWRR	Dynamic Wearout Resilient Routing
EM	Electromigration
F-APC	Forward Allowed-Path-Counter
FoN	Fault-on-Network
FTMR	Fault-Tolerant Minimal Routing
HCI	Hot-Carrier Injection
MTTF	Mean Time To Failure
NoC	Network-on-Chip
PV	Process Variation
R-APC	Reverse Allowed-Path-Counter
RCA	Regional Congestion Awareness
RL	Reinforcement Learning
VC	Virtual Channel

Acknowledgements

First and foremost, I would like to extend my sincerest thanks to my supervisor, Andre Ivanov. Without his guidance and support, I would not have been able to pursue this path.

I am also grateful to the members of my research group who have given me valuable input throughout all stages of my work. Ian Hill, Roozmehr Jalilian, and Raul Vazquez Guerrero have been wonderful members of the group; their support has been indispensible.

Chapter 1

Introduction

Modern chip multiprocessors (CMPs) integrate a large number of processing cores, caches, and memory components on a single die. As the number of cores increases, efficient communication between these components becomes a critical design challenge. Traditional bus-based or point-to-point interconnects do not scale well due to bandwidth limitations, contention, and wiring complexity. To address these issues, modern CMPs adopt Network-on-Chip (NoC) architectures [1], which organize on-chip communication as a packet-switched network composed of routers and links. This approach improves scalability, enables higher parallelism in communication, and provides structured support for routing data between distributed components [1]. As a result, the NoC plays a central role in determining system-level performance, such as latency, throughput, and energy efficiency.

Figure 1.1 illustrates the fundamental differences between traditional bus-based interconnects and modern NoC architectures. In a bus-based system (Fig. 1.1a), all components share a common communication medium, leading to contention and long data paths as the number of cores increases. In contrast, a NoC architecture (Fig. 1.1b) employs distributed routers and links, allowing for shorter data paths and reduced latency, even as the system scales to many cores.

While NoCs improve scalability and performance, they also introduce new reliability challenges. With continued technology scaling and increasingly demanding workloads, CMPs operate under high power densities and elevated temperatures for extended periods. These conditions accelerate device degradation mechanisms, making long-term reliability a primary concern [2]. Two dominant aging mechanisms in modern technologies are electromigration (EM) [3] and hot-carrier injection (HCI) [4]. Electromigration occurs in metal interconnects when sustained high current densities cause the gradual displacement of metal atoms, leading to void formation or wire thinning, and eventually open-circuit failures [3]. In contrast, HCI affects transistors,

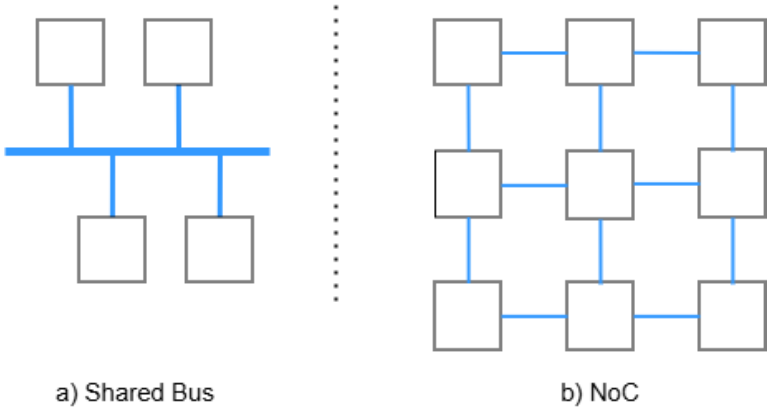


Figure 1.1: Comparison of traditional bus-based interconnect and modern NoC architecture in CMPs. (a) Bus-based interconnect with shared communication medium. (b) NoC architecture with distributed routers and links.

where high-energy carriers become trapped in the gate oxide, increasing the threshold voltage and degrading transistor performance over time [5]. Both mechanisms are strongly dependent on usage and temperature, meaning that frequently used components under high thermal stress degrade significantly faster than less utilized ones [2]. In addition to these wearout mechanisms, bias temperature instability (BTI) [4] can also contribute to aging in transistors in the presence of prolonged bias and elevated temperatures, resulting in threshold voltage shifts and performance degradation. However, since BTI is primarily induced when circuits are underutilized, it can be mitigated through techniques such as Use it or Lose it, as proposed by Kim et al. [6]. Therefore, this thesis focuses on EM and HCI as the primary aging mechanisms of concern in NoC components.

The impact of aging on NoC reliability is significant, as failures in critical components can lead to system-level failures, unlike core or cache failures that may be mitigated through redundancy or task mapping techniques [2]. Figure 1.2 illustrates different failure modes in a CMP, adapted from the conceptual model presented in [2]. As illustrated in Figure 1.2a), a core failure due to wearout may reduce computational capacity but does not necessarily lead to a complete system failure, as tasks can be remapped to other cores [7]. In contrast, a router failure (Fig. 1.2b) or a link failure (Fig. 1.2c) can disrupt communication between multiple components by severing critical paths and potentially isolating parts of the network. This

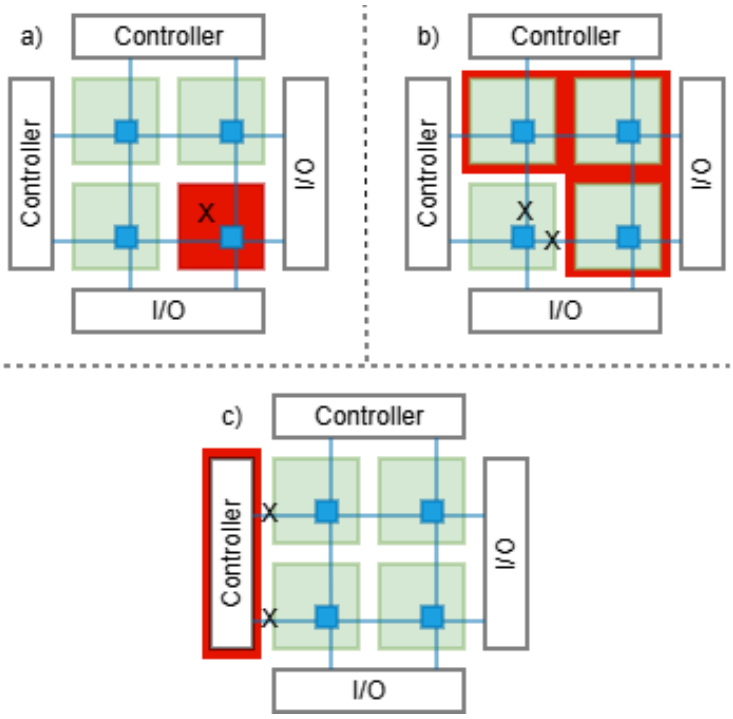


Figure 1.2: Component failures due to wearout in a 4-core CMP with a 2D mesh NoC interconnect. (a) Core failure. (b) Router failure. (c) Link failure, adapted from [2].

loss of connectivity can render multiple cores inaccessible [2]. Consequently, the reliability of NoC components is crucial for maintaining overall system functionality and performance over time.

Routing algorithms play a critical role in determining both the performance and reliability in NoCs. By controlling how traffic is distributed across the network, routing decisions directly influence the utilization and stress experienced by routers and links [8]. Conventional routing algorithms, such as dimension-order routing (DOR) [9], which follows deterministic path based on a fixed order of dimensions (e.g., X-Y), do not consider the dynamic state of the network or the aging effects on components. Their deterministic nature leads to uneven traffic distribution, with central routers and links experiencing consistently higher utilization. This imbalance translates directly into accelerated aging of specific components, effectively reducing the lifetime of the entire network. To address the limitations of deterministic

routing, adaptive routing algorithms have been proposed. These methods dynamically select packet routing based on network conditions, such as congestion, buffer occupancy, or link availability. By distributing traffic more evenly across the network, adaptive routing can improve both performance and load balancing [8]. Figure 1.3 illustrates the difference between deterministic and adaptive routing schemes in a 4x4 mesh NoC with data traversing from source router, A, to destination router, B.

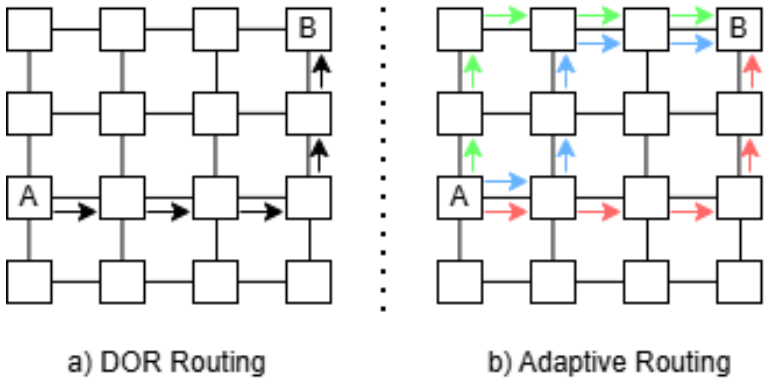


Figure 1.3: Routing algorithms for managing traffic in a 4x4 mesh NoC. (a) Dimension-order routing (DOR) with deterministic paths. (b) Adaptive routing with dynamic path selection based on network conditions.

1.1 Problem Statement

Prior research on NoC reliability has largely focused on fault-tolerant routing. In these approaches, the primary objective is to maintain connectivity after faults have already occurred. Fault-tolerant routing algorithms detect failed components and reconfigure paths to avoid them, ensuring continued operation despite permanent faults [10] [11] [12]. However, these methods are inherently reactive: they address failures only after they manifest, and therefore do not prevent or delay the underlying aging processes that lead to these faults [8]. In contrast, aging-aware routing adopts a proactive approach [2] [13]. Instead of reacting to failures, these algorithms aim to delay their occurrence by controlling how traffic is distributed over time [8]. Aging-aware routing leverages metrics such as utilization, temperature, and estimated wear to guide routing decisions, with the goal of balancing stress across network components. By avoiding persistent overuse of specific

routers or links, these techniques seek to extend the overall lifetime of the NoC. Like fault-tolerant methods, aging-aware routing is adaptive in nature, but its objective is fundamentally different: it optimizes long-term reliability rather than short-term fault avoidance.

Existing aging-aware routing techniques typically rely on explicit modeling of component reliability. For example, they may estimate the mean time to failure (MTTF) of links and routers based on EM and HCI models, and use these estimates to compute reliability-aware path costs [2]. Routing decisions are then made by selecting paths that minimize accumulated degradation. While effective in principle, these approaches often require global knowledge of network state and repeated evaluation of candidate paths. This introduces significant computational complexity, memory, and footprint overhead, making them less practical for large-scale systems [2] [14]. Moreover, their reliance on predefined cost functions and heuristics limits their ability to adapt to dynamic and evolving traffic patterns over long execution periods.

1.2 Scope of Research

This thesis addresses the limitations of existing aging-aware routing techniques by proposing a learning-based routing framework that dynamically adapts to both traffic conditions and progressive wear in NoC systems. Unlike prior approaches that rely on explicit fixed heuristic cost functions, the proposed method learns routing decisions directly through interaction with the network, enabling continuous adaptation without requiring global state information or repeated path evaluation. Similar to prior aging-aware routing work, the proposed approach aims to mitigate wearout by distributing traffic away from highly stressed components. To achieve this, the system monitors key stress indicators at the microarchitectural level, including switching activity and temperature, which directly influence aging mechanisms such as EM and HCI. These parameters are used in conjunction with established EM and HCI models to capture the evolving degradation state of routers and links over time.

Building on this information, the routing strategy is formulated as a reinforcement learning (RL) problem. Each router acts as an agent that learns to select output ports based on local observations of network state and stress conditions. To ensure scalability and low hardware overhead, the proposed method employs linear function approximation [15], allowing routing deci-

sions to be computed efficiently using a compact set of features rather than maintaining large state-action tables. Through this formulation, the learned policy implicitly captures the relationship between traffic distribution and long-term aging, enabling the system to avoid overusing vulnerable components without relying on explicitly defined cost metrics.

In addition to improving adaptability, the proposed approach addresses practical implementation constraints. Since adaptive routing can introduce cyclic dependencies and deadlock, the routing algorithm is integrated with a deadlock-free design using escape virtual channels [16]. This ensures correctness while preserving the flexibility of adaptive routing decisions.

The proposed routing framework is implemented within the gem5 Garnet NoC simulator, enabling cycle-accurate evaluation under realistic workloads. Experimental results demonstrate that the learned routing policy achieves improved lifetime characteristics by balancing stress across the network, while maintaining competitive performance in terms of latency and throughput.

The key contributions of this work are summarized as follows:

- A reinforcement learning-based aging-aware routing algorithm that dynamically adapts to traffic and wear conditions
- Integration of a linear function approximation model [15] that enables scalable, low-overhead implementation at the router level
- Integration of microarchitectural aging indicators, including EM [3] and HCI [4] related stress factors, into the routing decision process
- Integration of a deadlock-free adaptive routing design [16] using escape virtual channels
- Implementation and evaluation within the gem5 Garnet simulator [17] [18], demonstrating improved network lifetime and balanced wear distribution

Chapter 2

Literature Review

2.1 Introduction to Routing Algorithms in NoCs

Routing determines how packets traverse a NoC from source to destination. In a typical 2D mesh topology, each router forwards packets based on a routing function implemented within its routing unit. The primary objectives of routing are to minimize latency, maintain high throughput, and ensure correctness by avoiding deadlock and livelock.

NoC routing algorithms are broadly classified as deterministic or adaptive. Deterministic (static) routing follows fixed, predefined paths that do not depend on runtime conditions. A widely used example is XY routing [9], where packets are first routed along the X dimension and then along the Y dimension. This approach is simple and hardware-efficient, but it cannot respond to traffic imbalance, faults, or localized congestion, which may lead to performance degradation and uneven resource utilization [13].

To address these limitations, adaptive routing algorithms dynamically select paths based on the current state of the network. Routing decisions may incorporate local or global information, including buffer occupancy, link utilization, fault status, or thermal conditions. Depending on their objective and design, adaptive routing strategies can be categorized into several classes. Following the taxonomy presented in [8], adaptive routing can be broadly grouped into reactive, proactive, and application-specific approaches.

One important class of reactive adaptation is congestion-aware routing, which aims to improve performance by steering traffic away from congested regions of the network. These algorithms use runtime indicators such as queue lengths or buffer occupancy to balance load and reduce packet delay [8]. Another class of reactive routing is fault-tolerant routing, which ensures continued packet delivery in the presence of permanent faults in routers or links. These approaches detect failures and reroute traffic around

faulty components, maintaining network functionality after degradation has occurred [8]. In contrast, proactive routing approaches aim to mitigate degradation before it results in failure. Aging-aware routing falls into this category, where routing decisions are influenced by long-term stress indicators such as temperature and switching activity. By distributing traffic more evenly across the network, these methods seek to reduce the rate of wearout mechanisms [8]. In this thesis, we focus on adaptive routing algorithms. Specifically, we review the state of the art in fault-tolerant, congestion-aware, and aging-aware routing, and identify gaps that motivate our proposed wear-aware routing algorithm based on linear approximate reinforcement learning.

2.2 Congestion-Aware Routing Algorithms

A major branch of adaptive routing research in NoCs focuses on congestion awareness. The main objective of these algorithms is to reduce packet latency and improve throughput by steering traffic away from overloaded routers and links [8]. In contrast to deterministic routing methods such as XY routing, congestion-aware approaches exploit runtime information such as buffer occupancy, queue length, or downstream link status to make more informed path selections. The literature in this area generally evolved from lightweight local schemes, to approaches incorporating wider congestion visibility, and finally to learning-based methods that attempt to infer better routing decisions from experience.

DyAD [19] combines deterministic and adaptive routing in a lightweight manner. Under low-congestion conditions, it behaves like a deterministic routing algorithm, while under congestion it switches to odd-even adaptive routing using congestion flag wires derived from buffer occupancy. DyAD achieved a throughput improvement of about 61.7% over XY routing and reduced latency by 14% relative to odd-even routing. However, DyAD relies only on local congestion information. As a result, its decisions may be shortsighted, since a direction that appears uncongested locally may still lead to heavier contention farther downstream. To overcome the limited visibility of purely local schemes *Destination-Based Adaptive Routing* (DBAR) [20] augments local routing decisions with propagated congestion information from routers lying in the quadrant between the current router and the destination. DBAR improves average saturation throughput by 25.2% compared with traditional routing. Even so, DBAR still incurs extra state propagation and hardware complexity, and its effectiveness depends on how accurately

the propagated congestion information reflects current network conditions. A further refinement of this idea appears in *Regional Congestion Awareness* (RCA) [21]. RCA collects congestion information from a larger region of the network and uses that information to guide path selection more intelligently than purely local methods. The survey describes several RCA variants with different scopes of visibility, and reports latency reductions of 16–71% relative to dimension-order routing. It also highlights a recurring trade-off in congestion-aware routing: the more broadly congestion information is collected, the greater the risk of communication overhead, stale information, and increased router complexity.

While these classical congestion-aware algorithms improved performance, most of them still relied on explicitly designed heuristics. The router designer had to choose what congestion signals to monitor, how far to propagate them, and how to combine them into a routing decision. This heuristic dependence limits adaptability when traffic conditions vary significantly over time. It also motivates the use of reinforcement learning, where routers can learn routing behavior from experience rather than depending entirely on manually crafted decision rules.

An early step in this direction is the RL-based routing-selection framework discussed in the survey under references [22] and [23]. In this formulation, reinforcement learning is used not to directly choose an output port, but to select among routing strategies such as XY, north-last, and adaptive north-last based on the observed NoC state. The state includes metrics such as injected and received flits, while the reward is tied to throughput. However, the action space remains relatively coarse since the learner chooses among pre-existing routing algorithms rather than learning a more fine-grained router-level policy. A more direct learning-based approach is represented by *DRLAR* [24], which formulates routing as a multi-objective optimization problem and uses a lightweight distributed deep Q-network deployed at each router. Instead of merely switching among preset algorithms, DRLAR estimates Q-values from local network state and directly learns routing behavior that balances load, congestion, and endpoint isolation. DRLAR reduces average latency by 33.3% and improves saturation throughput by up to 20% over approaches such as Footprint routing. Still, deep RL introduces training cost and implementation overhead, which may limit practicality in hardware-constrained NoC routers. Another notable line of work is *HARAQ* [25], a Q-learning-based routing algorithm in which each switch maintains a Q-table containing estimated latency values for reaching destinations through different output channels. Packets are forwarded

through the channel with the lowest estimated cost, and the Q-values are updated using observed feedback. This approach results in latency reductions of up to 37% compared with methods such as DBAR. However, the state-action space grows rapidly with network size and traffic diversity, and convergence can be slow in dynamic environments.

This scalability problem motivates the use of function approximation. *linear approximate RL-based adaptive routing* [15], instead of storing a Q-value for every state-action pair, it estimates Q-values using a linear model over selected features. The reported result is a 10.5% reduction in packet latency compared with DOR and Footprint routing, while significantly reducing memory requirements relative to Q-table-based approaches. This idea preserves the adaptive nature of RL while making implementation more feasible for NoC routers with strict hardware budgets. In that sense, linear approximation forms a bridge between classical heuristic routing and more costly deep RL approaches, and it is especially relevant for architectures where low-overhead distributed decision-making is required.

2.3 Fault-Tolerant Routing Algorithms

Fault-tolerant routing has been a major area of research in NoC reliability because failures in routers or links can disrupt packet delivery, reduce throughput, and in severe cases partition the network. Unlike congestion-aware routing, which mainly targets performance, fault-tolerant routing is primarily concerned with preserving connectivity and ensuring successful packet delivery in the presence of permanent or intermittent faults [8]. In practice, these algorithms attempt to detect faulty components and reroute traffic through healthy regions of the network while maintaining acceptable latency and throughput.

One representative early approach is the *Fault-Tolerant Minimal Routing* (FTMR) [26] algorithm. FTMR seeks to preserve minimal routing whenever possible by identifying the shortest viable path around faults. It does this through two procedures, Forward Allowed-Path-Counter (F-APC) and Reverse Allowed-Path-Counter (R-APC), which respectively identify unreachable and useless nodes before routing decisions are made. The importance of FTMR lies in its explicit attempt to retain minimal-path behavior under fault conditions rather than immediately resorting to broad non-minimal detours. However, this benefit comes at the cost of additional computation, since the path analysis must be carried out in both forward and reverse

directions for each routing decision. As a result, FTMR is conceptually strong as a shortest-path fault-tolerant method, but less attractive when router simplicity and low overhead are primary design goals.

A more widely cited baseline in the fault-tolerant NoC literature is the *Fault-on-Neighbor* (FoN) [27] algorithm. FoN improves routing decisions by allowing a router to observe link status within a two-hop neighborhood and use that information to avoid nearby faults. The survey identifies FoN as a common reference point for evaluating later fault-tolerant routing schemes, which reflects its practical importance in the literature. Its appeal comes from the fact that it introduces fault awareness without requiring a fully global view of the network. At the same time, its reliance on local fault information also reveals a recurring limitation of many reactive fault-tolerant methods: a route that appears safe within a two-hop neighborhood may still lead to congestion, dead ends, or repeated rerouting farther downstream.

Another important direction in the literature is represented by the *Logic-Based Distributed Routing* [28] family, including LBDR, uLBDR, and d2-LBDR. These methods move away from large routing tables and instead use simple logic operations and configurable bits to encode connectivity and fault status. The survey reports that logic-based approaches can reduce area and power relative to table-driven methods, while still allowing routers to reconfigure paths around faults. The later d2-LBDR extension improves this framework further by maintaining the distance to the nearest known failure and incorporating that information into routing decisions. Its weakness, however, is that per-packet logic evaluation and dynamic reconfiguration can still become costly in large and highly dynamic NoC environments. The d2-LBDR framework also led to more advanced reconfiguration-oriented methods such as *TRACK* [29], which dynamically updates routing in response to directional link failures. TRACK achieves 75% reliability under double-link failures and improves latency and throughput by up to 42% and 22%, respectively, compared with existing fault-tolerant approaches. Still, the emphasis on nearby failures means that such methods may not fully capture more distant fault interactions along the route.

More recently, learning-based fault-tolerant routing has been explored as an alternative to fixed rerouting heuristics. The survey highlights a Q-learning-based routing algorithm in [11], where routers learn fault-tolerant forwarding choices by rewarding transitions that avoid unnecessary hops and improve packet delivery. The reported packet delivery reaches 95% in the presence of 10% faults, suggesting that reinforcement learning can

capture useful routing behavior even under degraded network conditions. A further extension in [12] applies multi-agent reinforcement learning, where routers make decentralized decisions using a two-level Q-table structure. The study reports a 7.1% improvement in packet delivery over the earlier Q-learning approach. These works are significant because they show that learning can improve adaptability in fault-tolerant routing, especially when fault patterns are irregular or difficult to encode using static rules. However, they also inherit the main limitation of slow convergence and growing state-action complexity, which may reduce effectiveness during early operation or in rapidly changing fault scenarios.

2.4 Aging-Aware Routing Algorithms

Aging-aware routing emerged as a proactive reliability strategy for Network-on-Chip systems, motivated by the observation that traffic distribution directly affects long-term wearout. Unlike fault-tolerant routing, which reacts after routers or links have already failed, aging-aware routing attempts to delay failure by controlling how stress accumulates across the network over time [8]. This is particularly important because aging mechanisms such as EM, HCI, and BTI are strongly influenced by switching activity, duty cycle, and temperature. As a result, heavily utilized routers and links tend to degrade faster than lightly used ones, making routing policy a critical determinant of NoC lifetime. In this body of work, the primary evaluation metric is typically mean time to failure (MTTF), which reflects the expected operational lifetime of the interconnect under a given routing strategy [8].

One of the clearest representative approaches in this area is the *stress-aware routing* algorithm in [2]. This method continuously monitors wearout risk in NoC components and routes traffic away from routers or links that are judged to be under higher stress due to temperature, utilization, or process-related effects. According to the study, the method improves MTTF by an average of 66% for EM risk and 8% for HCI risk relative to DOR. This result that lifetime extension can be achieved through traffic redistribution alone, without waiting for failures to appear. At the same time, the method depends on packet headers to disseminate network-state information, which means routing decisions may be affected by stale information when traffic conditions change quickly. In addition, the authors acknowledge that the method introduces significant hardware overhead due to the need for monitoring stress parameters.

Lifetime-Aware Routing (DPLR) [30], which introduces a lifetime budget for each router. The core idea is that every router has a maximum workload budget over a given period, and routing decisions should be made with respect to how much of that budget has already been consumed. The study reports that DPLR improves MTTF by 20–55% compared to conventional routing methods. However, it remains dependent on explicit cost modeling and repeated optimization, which can become difficult to sustain efficiently as the network grows or workload conditions become more dynamic. A more optimization-driven line of work is represented by the *dynamic programming (DP)-based aging-aware routing* approach in [31]. Rather than simply avoiding highly stressed components, this method explicitly computes routes that minimize the maximum aging rate across the network. In other words, it formulates routing as a constrained optimization problem whose objective is to produce a more uniform wear distribution and thereby extend overall NoC lifetime. The study reports that this approach improves MTTF by up to 20% over existing aging-aware routing techniques. However, like other dynamic programming methods, it can be computationally expensive and may struggle to converge rapidly in dynamic NoC environments where traffic conditions evolve continuously.

Another notable direction is runtime degradation tracking, represented by methods such as *Dynamic Wearout Resilient Routing* (DWRR) [32]. DWRR monitors router degradation through critical-path delay measurements and then broadcasts wearout information across the network so that traffic can be redistributed more evenly. The study reports a 25% improvement in MTTF over existing fault-tolerant techniques. However, this method introduces communication overhead, and in rapidly changing traffic conditions the propagated degradation information may already be outdated by the time it is used. A related example of this trade-off appears in the *centralized aging table* approach in [33]. In this method, a centralized structure tracks flit counts and residence times in routers to estimate temperature fluctuations, stress levels, and cumulative age. Routes are then selected according to the minimum cumulative aging cost from source to destination. The study reports a 39% improvement in degradation rate of aged routers and a 52% improvement in aging imbalance relative to traditional algorithms. However, it depends on centralized tracking, periodic updates, or globally maintained state, which reduces responsiveness and increases implementation overhead. In the case described in the survey, routing table updates occur only weekly, which may not accurately reflect more immediate changes in traffic-induced wear.

2.5 Motivation for Learning-Based Aging-Aware Routing

The literature on adaptive routing in NoCs shows a clear progression toward more flexible decision-making. In congestion-aware routing, early methods relied on fixed heuristics [19] [21], while more recent work extended this direction by introducing reinforcement learning and deep reinforcement learning [22] [23], allowing routers to adapt routing behavior based on observed network conditions. Similar trends can be observed in fault-tolerant routing, where learning-based methods such as Q-learning and multi-agent reinforcement learning were introduced to improve routing under irregular fault patterns and dynamic network degradation [11] [12]. These developments indicate that, in both congestion management and fault tolerance, the field has already begun moving beyond static heuristics toward policies that can be learned from interaction with the network. In contrast, aging-aware routing has not advanced as far in this direction. Existing aging-aware approaches successfully established that routing decisions can influence long-term reliability by redistributing traffic away from highly stressed routers and links [2] [30]. However, these approaches are still largely based on predefined models, hand-crafted cost functions, and explicit optimization procedures. Their adaptability therefore remains limited by the assumptions embedded in those models.

A recurring limitation in prior aging-aware routing research is the dependence on global or semi-global network knowledge. Several methods rely on packet-header-based dissemination of wear information, centralized aging tables, or broadcast-based monitoring of degradation state [2] [33]. While these mechanisms improve visibility into the aging condition of the network, they also introduce non-trivial communication, storage, and control overhead. More importantly, the information used for decision-making may become stale before it is applied [8]. As a result, many of these approaches are proactive in intent, but only partially adaptive in practice. Another important limitation is that most prior aging-aware routing algorithms treat the routing problem as an explicit optimization problem that must be repeatedly solved using predefined criteria [2]. This creates two challenges. First, the resulting routing policy is only as good as the chosen heuristic or optimization model. Second, the policy may not remain effective when traffic patterns, stress distributions, or workload characteristics change significantly over long execution periods. In other words, these methods are adaptive only within the boundaries of their manually defined objective

functions.

This stands in contrast to learning-based routing methods developed for congestion avoidance and fault tolerance [23] [11]. In those domains, reinforcement learning was introduced specifically to reduce dependence on manually engineered routing rules and to allow routing decisions to improve through experience. Q-learning [34], deep Q-networks [35], and linear function approximation [15] were all explored as ways to capture network behavior more flexibly and with different trade-offs in accuracy, scalability, and memory cost. The existence of these methods in adjacent routing domains suggests a natural next step for aging-aware routing. Rather than repeatedly computing routes from explicit aging models alone, routers can instead learn policies that implicitly capture the relationship between traffic distribution, network state, and long-term wear.

The key research gap, therefore, is not whether aging-aware routing is useful, since prior work has already shown that it is. The gap lies in how aging awareness is currently realized. Existing methods remain dominated by heuristic and model-driven formulations that incur hardware overhead, depend on explicit state propagation, and offer limited flexibility under changing operating conditions. Unlike congestion-aware and fault-tolerant routing, aging-aware routing has not yet been sufficiently extended into a lightweight learning-based framework that can adapt online while remaining practical for distributed router-level implementation. This gap motivates this thesis. The central idea is to move aging-aware routing from explicit heuristic optimization toward learning-based adaptation. By formulating routing as a reinforcement learning problem, the router can learn to avoid highly stressed NoC components through interaction with the network rather than by relying solely on predefined cost functions. At the same time, to remain practical for hardware implementation, such a method must avoid the large storage and convergence costs associated with conventional tabular learning. This makes linear approximate reinforcement learning [15] the perfect candidate, as it offers a balanced compromise between adaptability and implementation efficiency.

Chapter 3

Aging-Aware Routing Algorithm System Design

This section presents the system-level modeling framework and design choices used to enable reliability-aware routing in the NoC. The proposed approach integrates dynamic temperature variation, process variation, and aging models for EM and HCI directly into the routing decision process. These models are based on prior work in [2] and are combined with a reinforcement learning-based routing policy inspired by LARE [15].

Although other aging mechanisms, such as Bias Temperature Instability (BTI), also affect CMOS devices, they are not considered here. As discussed in [2], BTI degradation is primarily associated with underutilized circuits and can be mitigated using techniques such as *Use it or Lose it* [6]. In contrast, EM and HCI are driven by switching activity and temperature [2], making them closely tied to traffic patterns and more appropriate for routing-level optimization.

Unlike conventional RL routing algorithms that optimize only performance metrics such as latency and throughput, this framework incorporates device-level reliability into routing decisions. Temperature and utilization are modeled as dynamic runtime inputs, while process variation is treated as a static parameter determined during initialization.

3.1 Process Variation

Process variation (PV) arises from fabrication-time imprecision and results in deviations of transistor parameters from their nominal values. As CMOS technology scales, these variations become more pronounced and introduce spatial non-uniformity in device characteristics across the chip. In the context of NoC reliability, PV plays an important role in modulating aging behavior, particularly for HCI, where transistor-level characteristics directly

influence degradation rates [4].

Following the model described in [2], PV is captured by considering variations in two key transistor parameters: the threshold voltage (V_{th}) and the effective gate length (L_{eff}). For each router, these parameters are modeled as:

$$V_{th} = V_{th0} + \Delta V_{th}, \quad V_{th0} = 179 \text{ mV} \tag{3.1}$$

$$L_{eff} = L_{eff0} + \Delta L_{eff}, \quad L_{eff0} = 14.4 \text{ nm} \tag{3.2}$$

The deviations ΔV_{th} and ΔL_{eff} are composed of two independent components: systematic variation, and random variation [36].

$$\Delta V_{th} = \Delta V_{th,rand} + \Delta V_{th,sys} \tag{3.3}$$

$$\Delta L_{eff} = \Delta L_{eff,rand} + \Delta L_{eff,sys} \tag{3.4}$$

Systematic variation represents spatially correlated deviations caused by layout-dependent and process-wide effects [37]. Neighboring regions of the chip tend to exhibit similar variation values. Systematic variation is modeled using a multivariate normal distribution with a spherical spatial correlation structure [37]. The correlation between two regions depends only on their Euclidean distance r. The spherical correlation function is defined as [37]:

$$\rho(r) = \begin{cases} 1 - \frac{3r}{2\phi} + \frac{r^3}{2\phi^3}, & r \leq \phi \\ 0, & r > \phi \end{cases} \tag{3.5}$$

where ϕ represents the correlation range expressed as a fraction of the die length. In this work, $\phi = 0.5$, meaning that routers separated by more than half the die are effectively uncorrelated. Within a single region, variation is fully correlated ($\rho(0) = 1$), while distant regions exhibit no correlation [2].

Random variation, in contrast, is uncorrelated and occurs at a much finer granularity, reflecting device-level fluctuations such as dopant variation. Both random and systematic components are assumed to follow normal distributions with zero mean [36]. Since these components are independent, the total variation also follows a normal distribution, defined as [2]:

$$\sigma_{total} = \sqrt{\sigma_{rand}^2 + \sigma_{sys}^2} \tag{3.6}$$

The parameter values used in this work are taken directly from [2] and are summarized as follows:

- For V_{th}: $\sigma_{rand} = 6.3\%$, $\sigma_{sys} = 6.3\%$
- For L_{eff}: $\sigma_{rand} = 3.2\%$, $\sigma_{sys} = 3.2\%$
- Correlation range: $\phi = 0.5$

Process variation is treated as a static property determined at fabrication time. Accordingly, the values of V_{th} and L_{eff} are generated once during initialization and remain constant throughout execution. In the implemented model, each router is assigned a fixed pair of (V_{th}, L_{eff}) values, which are stored and used throughout the simulation.

To visualize the spatial effect of process variation across the NoC, Figure 3.1 illustrates example per-router maps of the generated threshold voltage and effective gate length values. Since the systematic component is spatially correlated, neighboring routers tend to exhibit similar deviations, producing smooth regional variation rather than entirely independent fluctuations. The random component is then superimposed locally, yielding the final per-router values used in the model. Since process variation is not uniform across the chip, different routers may begin operation with different intrinsic susceptibility to HCI induced degradation.

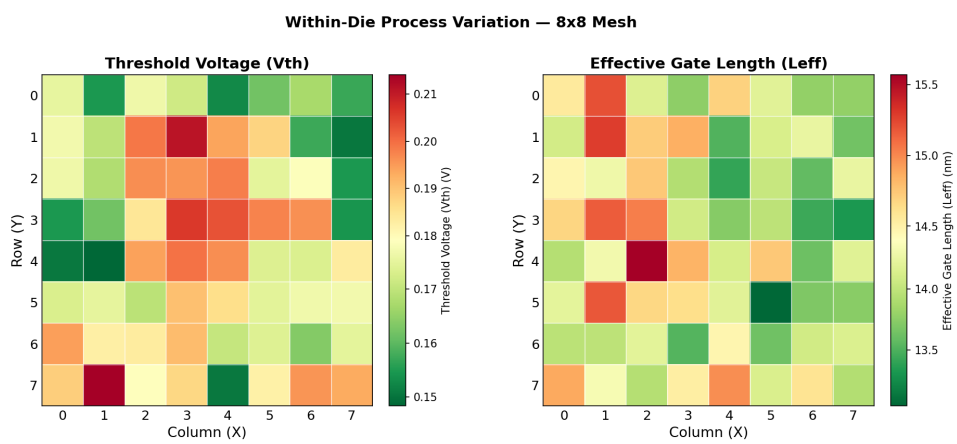


Figure 3.1: Example spatial mapping of process variation across the NoC: (left) threshold voltage, $V_{th0}+\Delta V_{th}$, and (right) effective gate length, $L_{eff0}+\Delta L_{eff}$, for all routers in the mesh. The maps reflect the combination of spatially correlated systematic variation and uncorrelated random variation applied at initialization.

The resulting per-router values of V_{th} and L_{eff} are subsequently used to

compute the substrate current (I_{sub}), which serves as an intermediate quantity in the HCI aging model. Since PV is static and all other parameters involved in this computation are technology constants, I_{sub} can be precomputed once at initialization. The substrate current is calculated as follows [38]:

$$I_{sub} = C_2 \cdot I_{ds} \cdot \exp\left(\frac{-\phi_i}{q\lambda_e E_m}\right) \tag{3.7}$$

where C_2 is a proportionality constant, I_{ds} is the drain current, ϕ_i denotes the minimum energy required for a hot electron to initiate impact ionization, q is the elementary charge, λ_e is the mean free path of hot electrons, and E_m is the electric field in the channel. To compute E_m, a semi-quntitative analytical model is used, expressed as [38]:

$$E_m = \frac{V_{ds} - V_{dsat}}{\sqrt{3t_{ox}x_j}} \tag{3.8}$$

where V_{ds} denotes the drain-to-source voltage and V_{dsat} corresponds to the voltage at the channel pinch-off point. The term $\sqrt{3t_{ox}x_j}$ represents a characteristic length associated with the effective thickness of the pinch-off region in the transistor channel. For typical CMOS technologies, this length falls within the range $\sqrt{100\text{ nm}}$ to $\sqrt{300\text{ nm}}$ [2].

For short channel devices, V_{dsat} is dependent on the effective gate length and can be modeled as [38]:

$$V_{dsat} = \frac{(V_{gs} - V_{th}) \cdot L_{eff} \cdot E_{cr}}{(V_{gs} - V_{th}) + L_{eff} \cdot E_{cr}} \tag{3.9}$$

where V_{gs} is the gate-to-source voltage, V_{th} is the threshold voltage, and E_{cr} denotes the critical electric field associated with velocity saturation.

All parameters other than V_{th} and L_{eff} are treated as technology-dependent constants and are fixed based on values reported in prior work [2]. The resulting I_{sub} value is stored as a single scalar per router, eliminating the need for repeated evaluation of the above expressions during runtime. The precomputed I_{sub} is later used in the HCI aging model to determine degradation rates as a function of switching activity and temperature.

3.2 Dynamic Temperature Variation

Temperature is one of the primary factors governing the rate of both EM and HCI induced aging, and is therefore modeled explicitly as a dynamic runtime parameter. In a CMP, temperature is not spatially uniform. Instead, it varies across the die due to non-uniform activity in the processing and memory resources, which arises from application mapping, workload imbalance, and differences in instruction behavior [39]. Following the modeling approach used in Clotho [2], the chip is partitioned into an $n \times n$ grid of equal-area tiles. Each tile represents a thermal region containing one processing core, its associated cache resources, and the locally connected router with its nearest-neighbor bidirectional links. Within each tile, temperature is assumed to be uniform, while different tiles may exhibit different temperatures. This assumption is justified by the observation that the NoC contributes only a relatively small fraction of the total chip power budget, and that this power is spatially distributed across the die rather than concentrated in a small number of locations [40]. Consequently, the dominant source of thermal variation is taken to be the computational activity of the processing elements, while the routers are treated as thermally coupled to the tiles that contain them [2].

Temperature is modeled as a slowly varying parameter rather than as a quantity that changes every cycle. In [2], the absolute values of the temperature map are updated every 10^7 cycles in order to emulate the gradually changing thermal behavior of a running CMP, consistent with prior observations by Mulas et al. [39], that on-chip temperature sensing and thermal evolution occur over millisecond-scale intervals rather than cycle-by-cycle timescales. The same assumption is adopted here. Accordingly, the temperature map is periodically refreshed every 10^7 cycles, allowing the routing algorithm to respond to long-term changes in the thermal profile without introducing unnecessary per-cycle thermal recomputation. The temperature assigned to each tile is generated as part of a spatial temperature map spanning the range of 65°C to 85°C, consistent with the range used in [2]. In implementation, these values are generated using a random but spatially correlated distribution so that neighboring tiles tend to exhibit similar thermal behavior. Figure 3.2 illustrates an example of such a temperature map. This per-tile temperature information is then used directly by the EM and HCI aging models. Since both failure mechanisms accelerate at higher temperatures, tiles operating at elevated temperature accumulate wear more rapidly than cooler tiles under otherwise similar activity conditions.

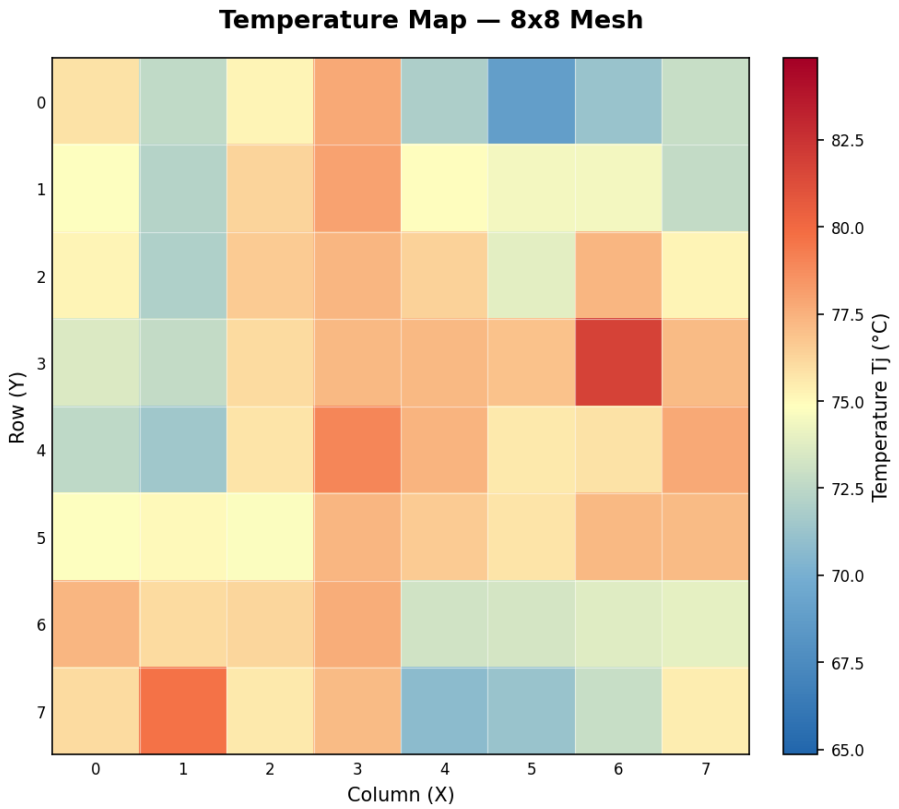


Figure 3.2: Example spatial mapping of temperature across the NoC. The map reflects the combination of spatially correlated thermal variation.

3.3 Aging Models

Electromigration (EM)

Electromigration (EM) is a wearout mechanism in which momentum transfer from conducting electrons causes gradual displacement of metal ions in interconnect wires [3]. Under sustained high current density, this ion migration leads to the formation of voids and hillocks, eventually resulting in open- or short-circuit failures. As technology scales and interconnect dimensions shrink, current densities increase, making EM a critical reliability concern in modern NoCs [2].

The lifetime of an interconnect under EM stress is commonly modeled using

Black's equation [3]:

$$MTTF_{EM} = A_{EM} J^{-n} \exp\left(\frac{E_{a,EM}}{kT}\right) \tag{3.10}$$

where J is the current density, n is an empirical exponent (typically close to 2 [2]), $E_{a,EM}$ is the activation energy, T is temperature, and k is Boltzman's constant.

Current density is closely related to switching activity [3]. As traffic increases, links are exercised more frequently, leading to a higher rate of current flow through the interconnect. Long router-to-router links tend to carry a larger share of traffic and exhibit higher effective current density compared to shorter intra-router wires. As a result, these links are more susceptible to electromigration-induced degradation.

Following [2], current density is expressed in terms of link switching activity:

$$J \propto \alpha_{link} \tag{3.11}$$

Substituting this relationship into Black's equation yields a simplified proportional model [2]:

$$MTTF_{EM}(\alpha_{link}, T) \propto \alpha_{link}^{-2} \cdot \exp\left(\frac{E_{a,EM}}{kT}\right) \tag{3.12}$$

The switching activity α_{link} is derived from link utilization [2]:

$$\alpha_{link} = 0.5 \times U_{link} \tag{3.13}$$

which assumes random data patterns, where each bit has a 50% probability of switching.

In this work, EM is modeled at the granularity of individual links. A proportional MTTF value is computed for each link based on its runtime utilization and temperature. These values are updated periodically as network conditions evolve.

A key characteristic of this model is its quadratic dependence on switching activity. As a result, EM is highly sensitive to traffic distribution. Even modest reductions in link utilization can produce significant improvements in lifetime. This makes EM particularly responsive to routing-level load balancing, where redistributing traffic away from heavily utilized links can substantially reduce localized wear.

Hot-Carrier Injection (HCI)

Hot-carrier injection (HCI) is a transistor-level aging mechanism caused by high-energy carriers that become trapped in the gate oxide during switching [4]. Over time, this leads to a gradual increase in threshold voltage (V_{th}), which slows transistor switching speed and degrades circuit performance. In NoC routers, this degradation is most critical along timing-critical paths, where increased delay can eventually result in timing violations and system failure [2].

The lifetime of a device under HCI stress can be expressed as [4]:

$$MTTF_{HCI}^{DC} = B_{HCI}(I_{sub})^{-N} \exp\left(\frac{E_{a,HCI}}{kT}\right) \tag{3.14}$$

where B_{HCI} is an arbitrary scale factor, I_{sub} is the substrate current during stress, N is an empirical exponent (approximately 1.5 [2]), T is the temperature, and $E_{a,HCI}$ is the activation energy.

Since HCI degradation occurs during switching events, its impact under dynamic operation is proportional to switching activity. Incorporating this effect yields the following model for HCI lifetime under switching activity [2]:

$$MTTF_{HCI}^{AC}(\alpha_{router}, T) \propto \frac{(I_{sub})^{-N}}{\alpha_{router}} \cdot \exp\left(\frac{E_{a,HCI}}{kT}\right) \tag{3.15}$$

Here, α_{router} represents the switching activity along the router's critical path. Based on empirical observations in [2], this activity is related to router utilization by:

$$\alpha_{router} = 0.1 \times U_{router} \tag{3.16}$$

reflecting that only a subset of flits (primarily header flits) exercise the critical path.

Unlike EM, HCI is strongly influenced by process variation. Variations in threshold voltage (V_{th}) and effective gate length (L_{eff}) affect the substrate current I_{sub}, which in turn determines degradation rate. As described in the previous section, I_{sub} is computed per router as a function of these parameters and treated as a static quantity throughout execution.

Combining these effects yields the final HCI model used in this work, following the formulation in [2]:

$$MTTF_{HCI,PV} \propto \frac{(I_{sub}(V_{th}, L_{eff}))^{-N}}{\alpha_{router}} \cdot \exp\left(\frac{E_{a,HCI}}{kT}\right) \tag{3.17}$$

This model captures both dynamic and static sources of degradation. The switching activity and temperature introduce time-varying stress, while process variation establishes a fixed baseline reliability for each router. As a result, routers with unfavorable (V_{th}, L_{eff}) values begin operation with reduced timing margin and are inherently more vulnerable to HCI-induced degradation.

Compared to EM, HCI exhibits a weaker dependence on switching activity due to its linear relationship with α_{router}. Consequently, while traffic redistribution can still improve HCI lifetime, its impact is less pronounced than in the case of EM.

Utilization of Lifetime Models

The EM and HCI models described above are used both to guide routing decisions and to evaluate system reliability. During routing, the MTTF of candidate next-hop links and routers is computed using recent utilization statistics and current temperature values. These MTTF estimates are normalized relative to the network state and used to define the reward signal for the reinforcement learning policy. As a result, routing decisions are biased toward paths that traverse components with higher estimated lifetime.

To capture current stress conditions, utilization is computed using a windowed formulation rather than cumulative statistics. Every 10^3 cycles, as opposed to 10^4 cycles proposed by [2], link and router activity is sampled to obtain a recent utilization rate, defined as the number of events observed within the window divided by the window duration. This produces a measure of instantaneous switching activity that reflects recent traffic behavior.

Using cumulative averages would dilute recent activity with historical behavior. For example, a component that was heavily utilized earlier in the simulation but is currently idle would still appear moderately stressed when using lifetime-averaged statistics. This leads to inaccurate MTTF estimates, as the model would continue to penalize components that are no longer under significant stress. In contrast, the windowed formulation ensures that utilization reflects the current operating conditions of each component. This is consistent with the underlying physics-of-failure models for EM and HCI, where degradation rates depend on the present level of electrical and thermal stress rather than long-term averages. As a result, the routing policy can more accurately identify and avoid currently stressed components.

The choice of a 10^3 cycle window provides a balance between responsiveness

and stability. Smaller windows can introduce noise due to short-term traffic fluctuations, while larger windows may fail to capture changes in load distribution in a timely manner. This design enables the routing policy to respond effectively to dynamic traffic conditions while maintaining stable utilization estimates.

Throughout simulation, these utilization snapshots are updated periodically and used to recompute EM and HCI MTTF estimates as needed. The MTTF values are not maintained as persistent aging state variables; instead, they are derived on demand from the latest utilization statistics, temperature map, and static process-variation parameters. This enables the routing policy to continuously adapt to dynamic traffic conditions. At the end of simulation, the final utilization statistics are used to compute MTTF values for all routers and links. The system lifetime is then defined by the most degraded component, reflecting the fact that failure of a single critical link or router can render the network unusable.

To compare routing algorithms, the acceleration factor (AF) is used as a relative lifetime metric, as defined by [4]:

$$AF_{EM} = \frac{MTTF_{EM}|_{custom_algo}}{MTTF_{EM}|_{DOR}}, \quad AF_{HCI} = \frac{MTTF_{HCI,PV}|_{custom_algo}}{MTTF_{HCI,PV}|_{DOR}} \tag{3.18}$$

Since both configurations share identical process variation and technology parameters, unknown constants and the precomputed I_{sub} cancel out in the ratio. As a result, the acceleration factor depends only on runtime variables such as utilization and temperature.

3.4 Reinforcement Learning-Based Routing Policy

Reinforcement learning (RL) [41] is a learning framework in which an agent interacts with an environment to learn a policy that maximizes long-term cumulative reward. At each decision step, the agent observes the current state s, selects an action a, and receives a reward r based on the outcome of that action. The system then transitions to a new state s', and the process repeats. Over time, the agent improves its decision-making by learning from these interactions. Figure 3.3 illustrates this agent–environment interaction model.

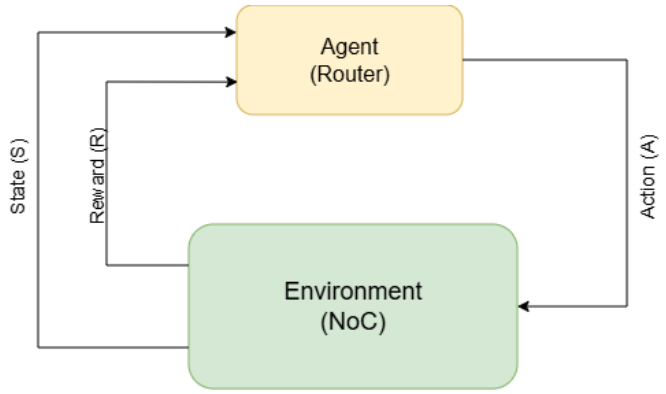


Figure 3.3: Agent–environment interaction in reinforcement learning.

In this work, we adopt Q-learning [34], a value-based RL method. Q-learning estimates the expected cumulative reward of taking an action a in a state s, represented by the Q-value function $Q(s, a)$. The optimal policy is obtained by selecting actions that maximize this value.

The Q-values are updated iteratively using the Bellman equation [42]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \tag{3.19}$$

where α is the learning rate and γ is the discount factor. The term inside the brackets is the temporal-difference (TD) error, which measures how well the current estimate matches observed outcomes.

In the context of NoCs, routing decisions can be formulated as a sequential decision-making problem, where each router selects the next-hop direction for a packet based on local network conditions. Each router is modeled as an independent RL agent, and routing corresponds to selecting an action that maximizes long-term reward [15]. Direct application of tabular Q-learning, however, is impractical in NoCs due to the large state-action space, which grows rapidly with network size. This leads to excessive storage requirements and slow convergence, commonly referred to as the curse of dimensionality [25]. To address this limitation, this work adopts a linear function approximation approach inspired by LARE [15], where Q-values are represented using a compact feature-based model instead of a lookup table.

State Representation

Routing decisions are made at each router when multiple minimal paths are available. Instead of representing the state explicitly, each state-action pair (s, a) is encoded using a feature vector $\phi(s, a)$, adapted from [15]:

$$\phi(s, a) = [f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7, f_8] \tag{3.20}$$

where:

- f_0: Bias term (constant 1)
- f_1: Destination router ID
- f_2: Current router ID
- f_3: Distance traveled in hops from source to current router
- f_4: Remaining distance to destination
- f_5: EM MTTF of candidate link
- f_6: HCI MTTF of candidate router
- f_7: Output port congestion level
- f_8: Encoded input port direction

This representation allows the agent to incorporate both performance related and reliability related information into routing decisions.

Q-Value Function

To reduce hardware overhead, the Q-value is approximated using a linear function [15]:

$$Q(s, a) = \theta_0 \cdot f_0 + \theta_1 \cdot f_1 + \theta_2 \cdot f_2 + \theta_3 \cdot f_3 + \theta_4 \cdot f_4 + \theta_5 \cdot f_5 + \theta_6 \cdot f_6 + \theta_7 \cdot f_7 + \theta_8 \cdot f_8 \tag{3.21}$$

where f_i represents the i-th feature in the state-action vector, and θ_i is the corresponding learned coefficient. Each coefficient captures the contribution of its feature toward the estimated long-term reward.

This approach eliminates the need to store a full Q-table, reducing memory requirements and enabling scalable implementation at the router level, while

still allowing the model to capture relationships between features and long-term reward.

Action Selection Policy

An ϵ-greedy policy is used to balance exploration and exploitation:

$$a = \begin{cases} \text{random action}, & \text{with probability } \epsilon \\ \arg\max_a Q(s,a), & \text{otherwise} \end{cases} \tag{3.22}$$

The action space consists of minimal routing directions (e.g., horizontal and vertical moves in a mesh topology).

Reward Function

The reward function is designed to jointly capture reliability and performance objectives. It combines normalized aging metrics with congestion information:

$$R = 0.7 \cdot \text{MTTF}_{score} - 0.3 \cdot \text{cong}_{norm} \tag{3.23}$$

The reliability component is defined as a weighted sum of normalized EM and HCI MTTF values, adapted from [2]:

$$\text{MTTF}_{score} = C_{EM} \cdot MTTF_{EM_{norm}} + C_{HCI} \cdot MTTF_{HCI_{norm}} \tag{3.24}$$

The EM and HCI values are normalized across candidate actions:

$$MTTF_{EM_{norm}} = \frac{MTTF_{EM}}{\max(MTTF_{EM}^{(h)}, MTTF_{EM}^{(v)})} \tag{3.25}$$

$$MTTF_{HCI_{norm}} = \frac{MTTF_{HCI}}{\max(MTTF_{HCI}^{(h)}, MTTF_{HCI}^{(v)})} \tag{3.26}$$

where the superscripts (h) and (v) denote the horizontal and vertical candidate next hops, respectively.

To dynamically emphasize the more critical aging mechanism, and to encourage the agent to route packets around heavily stressed components, adaptive weights are computed using the model proposed in [2]:

$$C(x) = \frac{\exp\left(8 \cdot \frac{x - x_{max}}{x_{max} - x_{min}}\right) - \exp(-8)}{1 - \exp(-8)} \tag{3.27}$$

where $x \in \{MTTF_{EM}, MTTF_{HCI}\}$, and x_{max} and x_{min} are the maximum and minimum values among candidate actions. The weights are then defined as:

$$C_{EM} = C(MTTF_{EM}), \quad C_{HCI} = C(MTTF_{HCI}) \tag{3.28}$$

The congestion term reflects the load on the output port:

- Low congestion: 0
- Medium congestion: 0.5
- High congestion: 1.0

This formulation encourages the routing policy to prefer paths that both reduce congestion and avoid highly stressed components.

Update Rule

The parameter vector θ is updated using temporal-difference learning. The update rule for linear function approximation is given by [15]:

$$\theta \leftarrow \theta + \alpha \cdot \delta \cdot \phi(s, a) \tag{3.29}$$

where δ is the TD error defined as:

$$\delta = R + \gamma \max_{a'} Q(s', a') - Q(s, a) \tag{3.30}$$

This update adjusts the model parameters in the direction that reduces the discrepancy between predicted and observed rewards, allowing the routing policy to improve over time.

3.5 Deadlock Avoidance

Adaptive routing algorithms, including the proposed RL-based routing policy, do not inherently guarantee deadlock freedom [15]. Since routing decisions are made dynamically based on network conditions, packets may acquire resources in different orders, potentially leading to cyclic dependencies. To understand how deadlock arises and how it can be prevented, it is first necessary to examine the underlying router microarchitecture.

Router Microarchitecture

A typical NoC router employs virtual channel (VC) flow control to efficiently utilize network resources. As shown in Figure 3.4, each router consists of input ports with associated buffers, a routing unit, a switch allocator, a crossbar, and output ports.

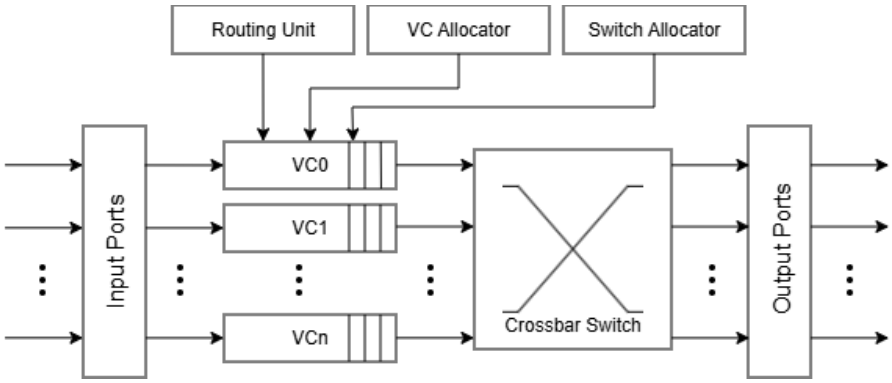


Figure 3.4: Router microarchitecture based on virtual channel flow control, adapted from [8].

Incoming packets arrive at an input port and are stored in input buffers organized as multiple virtual channels (VC) [43]. Each VC maintains a separate queue of flits, allowing multiple packets to share the same physical channel without blocking each other. When a packet reaches the head of a VC, the routing unit determines the set of candidate output ports based on the routing algorithm [1]. The packet then requests access to a downstream VC at the next router. Once a downstream VC is successfully allocated, the packet competes for access to the crossbar through the switch allocator [8]. If granted, the packet is forwarded through the crossbar to the selected output port and transmitted to the next router [44]. This process repeats

at each hop, resulting in a sequence of resource acquisitions: input VC, downstream VC, switch allocation, and output link traversal.

Deadlock Formation

The staged allocation of resources creates dependencies between packets across routers. A packet occupying a VC at one router may be blocked while waiting for a downstream VC at the next router. If that downstream VC is already occupied by another packet, the dependency extends further along the network. Deadlock occurs when a set of packets forms a cyclic dependency, where each packet holds a resource while waiting for another resource held by another packet in the cycle [16].

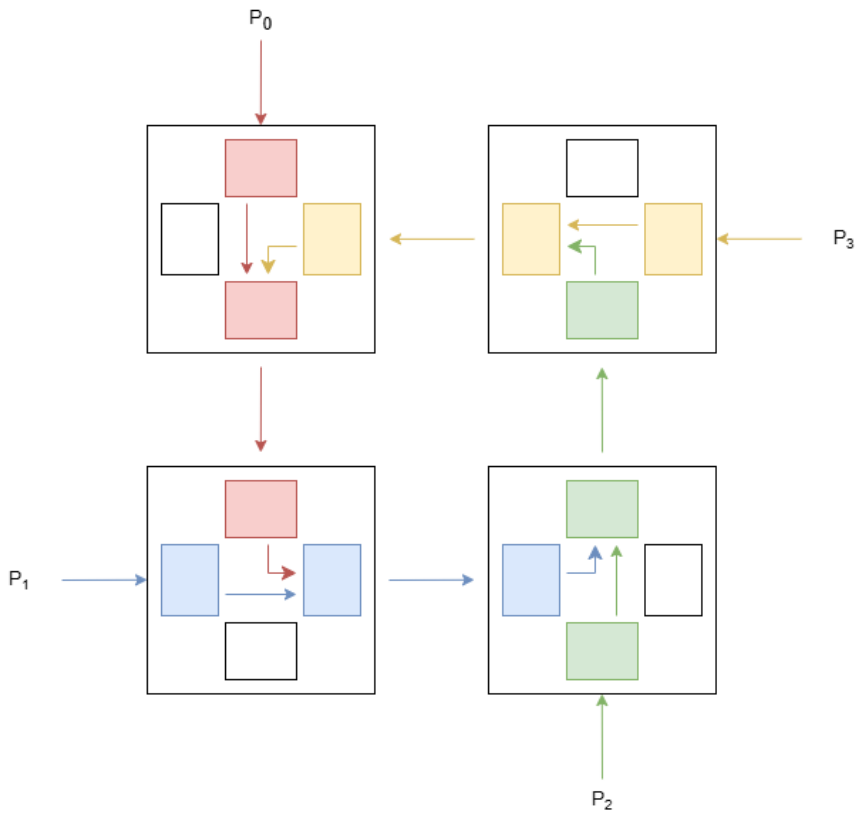


Figure 3.5: Example of cyclic resource dependency leading to deadlock in a mesh NoC, adapted from [45].

In a 2D mesh, such cycles can occur when packets routed in different di-

rections occupy VCs in neighboring routers while waiting for each other's downstream resources, forming a closed loop. Since none of the packets can release their current resources until they acquire the next one, no forward progress is possible [16]. This is illustrated in Figure 3.5, where packets P_0, P_1, P_2, and P_3 form a cycle of dependencies across four routers. Each packet is waiting for a buffer in the downstream router that is currently being occupied by another packet in the cycle, resulting in a situation where no packet can proceed and the system is effectively stalled.

Adaptive routing increases the likelihood of such cycles because it allows packets to dynamically select different paths [15]. Unlike deterministic routing, which enforces a strict ordering of channel acquisition, adaptive routing introduces flexibility that can lead to arbitrary dependency patterns and potential deadlock.

Deadlock Avoidance Using Escape Virtual Channels

To guarantee deadlock freedom, this work adopts an escape virtual channel mechanism based on Duato's theory [16]. The key idea is to reserve a subset of network resources that follow a strictly deadlock-free routing function, ensuring that at least one acyclic path is always available.

In this implementation, one virtual channel (VC0) is designated as the escape channel and is restricted to deterministic dimension-order (XY) routing [16]. Since XY routing enforces a strict ordering of channel acquisition and is provably deadlock-free [46], any packet assigned to this channel is guaranteed forward progress. All remaining virtual channels are allowed to use the RL-based adaptive routing policy. If a packet cannot safely proceed using adaptive routing, it can transition to the escape VC and continue using XY routing. This mechanism ensures that cyclic dependencies are avoided while preserving the flexibility of adaptive routing for the majority of traffic.

Chapter 4

Evaluation

4.1 Simulation Setup

All simulations are conducted using the gem5 Garnet cycle-accurate simulator [17] [18] in standalone synthetic traffic mode, without a full CPU or ISA model, to isolate network behavior. The network consists of 64 nodes arranged in an 8×8 2D mesh topology. Each node acts as both a traffic source and destination, with packets injected according to configurable synthetic traffic patterns at a specified rate in packets per cycle per node.

The simulated clock frequency is 1 GHz. The router microarchitecture is an input-queued, virtual-channel router with credit-based flow control. Each router has a pipeline latency of 1 cycle and each link has a traversal latency of 1 cycle. The network supports three virtual networks per input port. Each virtual network contains 4 virtual channels. Control virtual channels have a buffer depth of 1 flit, while data virtual channels have a buffer depth of 4 flits. Control messages consist of a single flit, whereas data messages consist of 5 flits. The flit width is 128 bits (16 bytes).

This work evaluates three routing algorithms: deterministic dimension-order routing (DOR) [47], the Clotho-GAR routing approach proposed by Vitkovskiy et al. [2], and the proposed RL-based routing policy. The Clotho framework is implemented as described in the original work to provide a direct comparison against a state-of-the-art aging-aware routing algorithm.

Unlike conventional RL workflows that separate training and evaluation phases, the proposed routing policy is learned online during simulation. Each run is executed for 50 million cycles, during which the RL agent continuously updates its parameters while routing packets. This allows the policy to adapt dynamically to traffic conditions and evolving network state without requiring a dedicated training phase.

The learning rate (α), exploration rate (ϵ), and discount factor (γ) are set to

0.01, 0.1, and 0.9, respectively [15]. The exploration rate remains constant throughout simulation, allowing continued exploration of routing decisions while gradually refining the policy. All routing algorithms are evaluated across a range of injection rates and synthetic traffic patterns. Each configuration is simulated for 50 million cycles.

The wearout state used in the RL reward is updated periodically during simulation. Link and router utilization are sampled every 1,000 cycles using a sliding window average. The spatial temperature map is regenerated every 10^7 cycles to reflect dynamic thermal variation. Process variation parameters (ΔV_{th} and ΔL_{eff}) are sampled once at initialization, ensuring that all routing algorithms operate under identical per-router I_{sub} values.

4.2 Garnet Synthetic Traffic and Evaluation Metrics

To evaluate the proposed routing approach under diverse network conditions, simulations are conducted using seven synthetic traffic patterns [48]: uniform random, neighbor, transpose, tornado, shuffle, bit reverse, and bit rotation. These patterns generate a wide range of communication behaviors, resulting in different load distributions across routers and links, and consequently different aging and congestion profiles.

The following traffic pattern description are extracted [48]. Uniform random traffic selects destinations randomly across all nodes, producing a balanced load throughout the network. Neighbor traffic restricts communication to adjacent nodes in the x-direction, resulting in highly localized traffic. Transpose traffic maps each node at position (i, j) to (j, i), creating structured long-distance communication. Tornado traffic sends packets from node (i, j) to a node offset by approximately half the network dimension, generating high contention in specific regions of the mesh. Shuffle, bit reverse, and bit rotation are permutation-based traffic patterns derived from bitwise transformations of node indices, producing structured yet asymmetric traffic flows that stress different regions of the network.

For each traffic pattern, injection rates are swept to capture both low-load and saturation regimes. For uniform random and neighbor traffic, injection rates range from 0.01 to 0.40 packets/cycle/node in increments of 0.03. For the remaining traffic patterns, injection rates range from 0.01 to 0.20 in increments of 0.01. Each configuration is simulated for 50 million cycles to

ensure sufficient observation of both transient and steady-state behavior.

The evaluation considers both performance and reliability metrics. Performance is measured using average packet latency. Reliability is evaluated using the acceleration factor (AF) derived from the EM and HCI lifetime models. AF measures the relative improvement in MTTF under a given routing algorithm compared to the DOR baseline. By combining performance and reliability metrics, the evaluation captures the trade-offs between network efficiency and long-term wear.

4.3 Results

Acceleration Factor

To summarize the reliability improvements across traffic patterns, the mean Acceleration Factor (AF) for the weakest 10% of components is presented using bar charts (Figures 4.1 and 4.2). Detailed AF trends as a function of injection rate are provided in Appendix A.

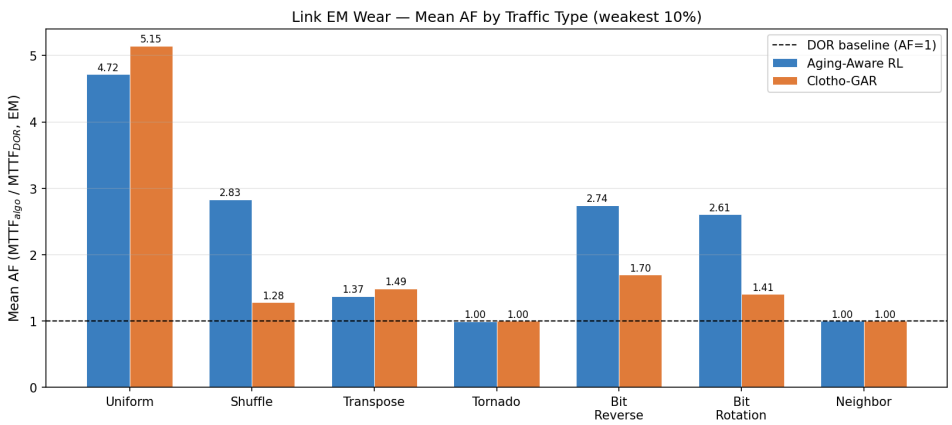


Figure 4.1: Mean Acceleration Factor (AF) for the weakest 10% of links under EM wear across different traffic patterns.

Overall, both the proposed RL-based routing and the Clotho aging-aware routing approach achieve significant improvements in lifetime compared to DOR. As expected, the improvement is more pronounced for EM than for HCI due to the quadratic dependence of EM lifetime on switching activity. Redistributing traffic away from heavily utilized links therefore yields substantial gains in EM lifetime, whereas HCI, with its linear dependence and

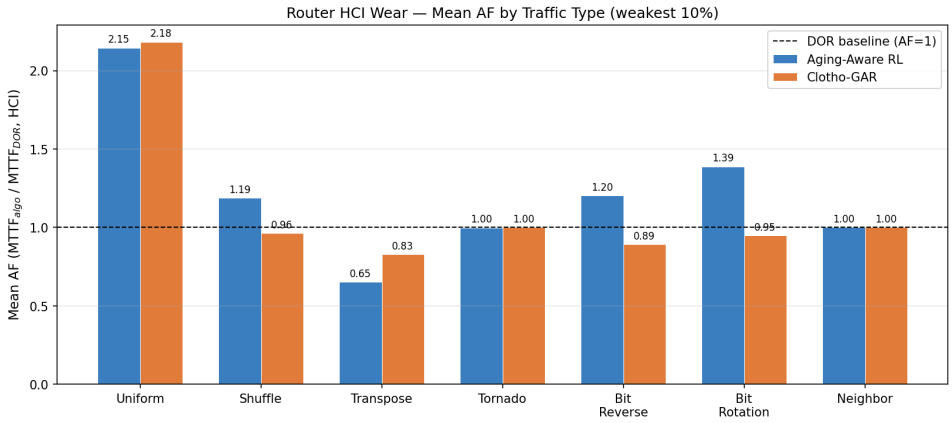


Figure 4.2: Mean Acceleration Factor (AF) for the weakest 10% of routers under HCI wear across different traffic patterns.

sensitivity to process variation, exhibits more modest improvements.

Electromigration (EM) Results For link-level EM wear, both the proposed RL-based routing and the Clotho-GAR approach significantly outperform DOR under several traffic patterns. Under uniform random traffic, both methods achieve substantial improvements, with Clotho reaching approximately 5.15× and RL achieving 4.72× improvement in MTTF. A similar trend is observed for transpose traffic, where both approaches provide modest improvements of approximately 1.4×. However, the performance diverges under structured and asymmetric traffic patterns. For shuffle traffic, RL achieves a 2.83× improvement, significantly outperforming Clotho, which provides only 1.28×. Similar trends are observed for bit reverse (2.74× vs. 1.70×) and bit rotation (2.61× vs. 1.41×). In contrast, both approaches show no improvement over DOR for tornado and neighbor traffic, indicating that these patterns either inherently balance load or offer limited routing flexibility.

These results highlight a key distinction between the two approaches. Clotho, which relies on predefined heuristics and explicit aging models, performs strongly under traffic patterns that align with its assumptions, such as uniform random traffic. However, its effectiveness degrades under traffic patterns with more complex or asymmetric load distributions. In contrast, the RL-based approach maintains strong performance across a wider range of patterns, demonstrating its ability to adapt routing decisions based on

observed network conditions.

Hot-Carrier Injection (HCI) Results A similar trend is observed for router-level HCI wear, although the magnitude of improvement is smaller due to the weaker dependence of HCI on switching activity. Under uniform traffic, both RL and Clotho achieve comparable improvements of approximately 2.15×.

Under transpose traffic, both approaches perform worse than DOR, with RL achieving 0.65× and Clotho 0.83×, indicating that traffic redistribution in this case introduces additional stress on critical paths. This highlights the sensitivity of HCI to routing decisions that may inadvertently concentrate activity on specific routers. For shuffle, bit reverse, and bit rotation traffic, the RL-based approach consistently outperforms both DOR and Clotho. In shuffle traffic, RL achieves 1.19× improvement, while Clotho degrades to 0.96×. Similar behavior is observed for bit reverse (1.20× vs. 0.89×) and bit rotation (1.39× vs. 0.95×). These results indicate that Clotho's heuristic-based routing can inadvertently increase stress under certain traffic patterns, while RL is able to avoid such degradation.

Discussion These results highlight two key observations. First, the effectiveness of adaptive routing is strongly dependent on path diversity. Traffic patterns that allow multiple routing choices benefit the most from load redistribution, while patterns with constrained paths show limited improvement.

Second, the RL-based routing approach demonstrates greater robustness across traffic patterns compared to Clotho. While Clotho performs well under traffic conditions that align with its heuristic design, its performance degrades under asymmetric or complex traffic patterns. In contrast, the RL-based approach maintains consistent improvements by adapting its routing decisions during execution. This behavior can be attributed to the fundamental difference in how the two approaches make routing decisions. Clotho relies on fixed heuristic models to evaluate path reliability, which limits its ability to adapt when traffic patterns deviate from expected behavior. The RL-based approach, on the other hand, continuously updates its policy during execution, allowing it to learn and adapt to the current traffic distribution and evolving network state. As a result, the RL-based routing policy is better able to respond to varying and evolving traffic patterns, leading to more effective stress distribution and improved overall reliability.

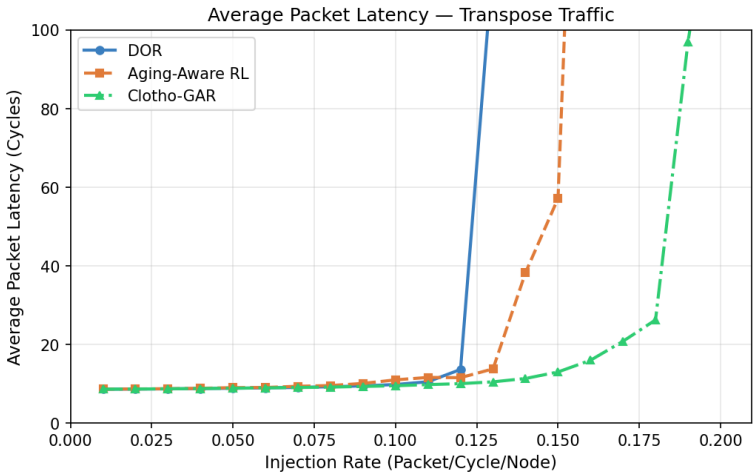
Latency Analysis

To evaluate the performance impact of the proposed routing algorithm, average packet latency is measured as a function of injection rate across all traffic patterns. Representative latency curves are shown in Figures 4.3, 4.4, 4.5, and 4.6.

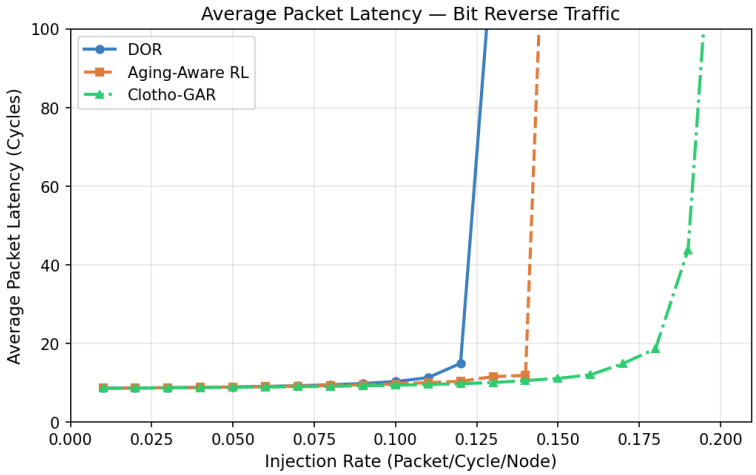
Across all traffic patterns, both the RL-based routing and the Clotho-GAR approach achieve latency comparable to or better than the DOR baseline. At low injection rates, all routing algorithms exhibit nearly identical latency, as the network is lightly loaded and contention is minimal. At higher injection rates, the benefits of adaptive routing become more apparent. DOR saturates earlier due to its deterministic nature, which concentrates traffic along fixed paths and creates congestion hotspots. In contrast, both RL and Clotho distribute traffic more evenly across available paths, delaying the onset of congestion and maintaining lower latency under higher load conditions.

For most traffic patterns, including uniform random, shuffle, bit rotation, neighbor, and tornado, RL and Clotho achieve similar latency performance. In these cases, both approaches effectively balance load and avoid congestion, resulting in comparable improvements over DOR. However, differences emerge under certain structured traffic patterns. In transpose and bit reverse traffic, Clotho achieves slightly lower latency than RL at higher injection rates. This indicates that Clotho's precomputation strategy allows it to more effectively minimize congestion in these scenarios. These observations highlight an important trade-off. Under bit reverse traffic, RL achieves significantly higher reliability improvement compared to Clotho, as shown in the AF results, but incurs a slight latency penalty. Similarly, under transpose traffic, RL improves link wear relative to DOR but performs slightly worse than Clotho in both latency and overall reliability metrics.

Despite these localized differences, the RL-based routing approach maintains competitive latency across all traffic patterns without significant degradation. Its performance remains close to Clotho while providing more consistent reliability improvements across diverse traffic conditions. Overall, these results demonstrate that RL-based routing is able to achieve a balanced trade-off between performance and reliability. While heuristic-based approaches such as Clotho may outperform RL under specific traffic patterns, RL offers more robust performance across varying and unpredictable traffic conditions, making it better suited for dynamic NoC environments.



(a) Average Packet Latency under Transpose Traffic.



(b) Average Packet Latency under Bit Reverse Traffic.

Figure 4.3: Average Packet Latency vs. Injection Rate for RL-based Routing and DOR across Transpose, Bit Reverse, and Bit Rotation traffic patterns.

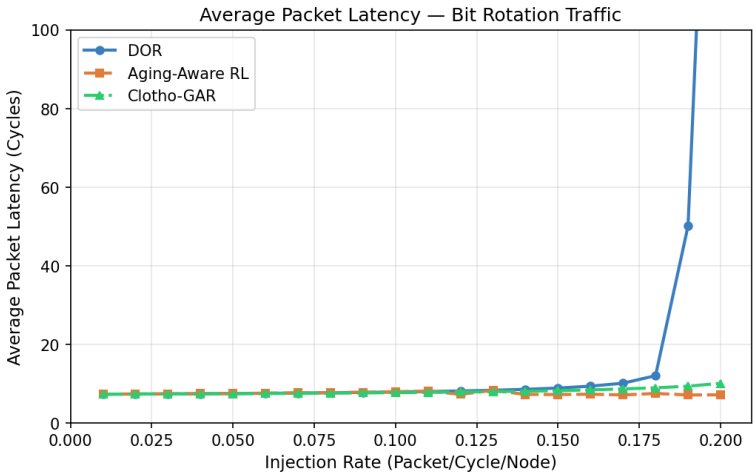
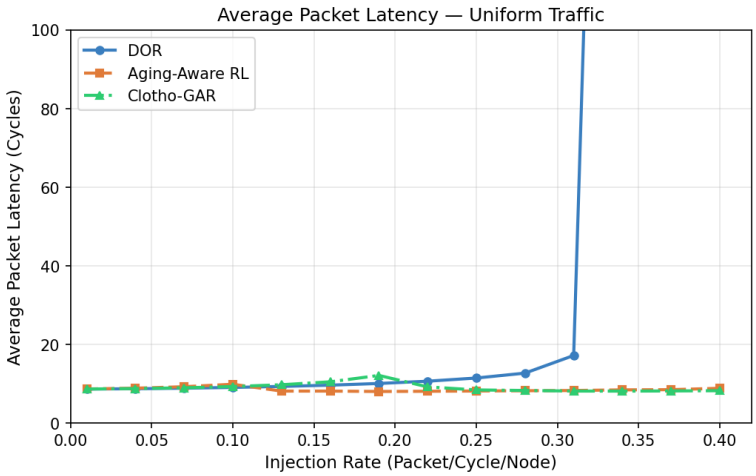
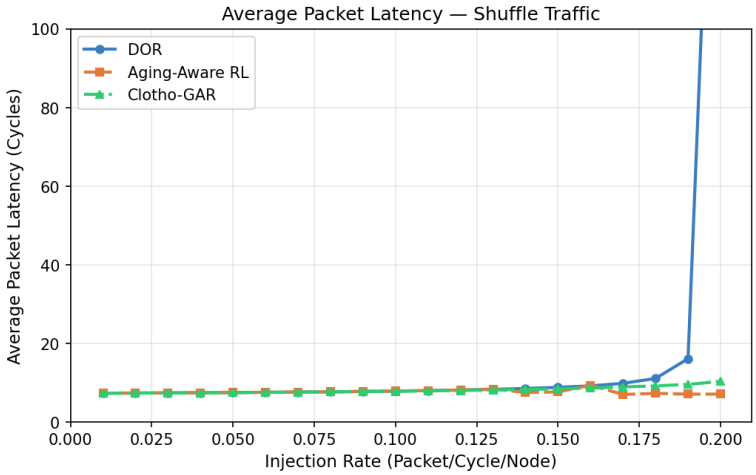


Figure 4.4: Average Packet Latency under Bit Rotation Traffic.

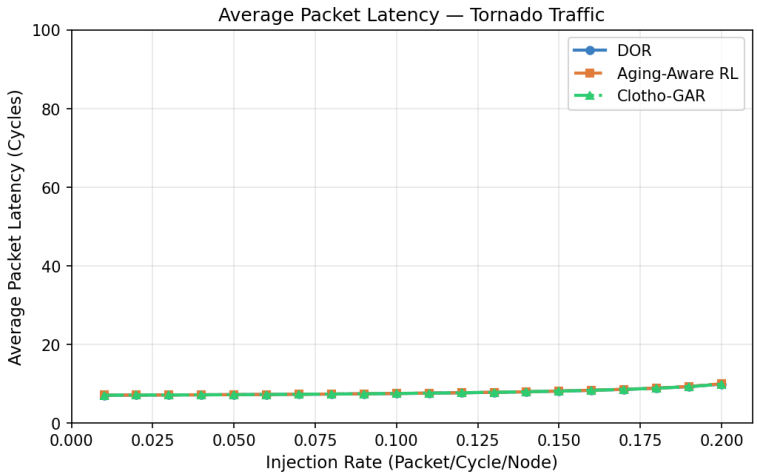


(a) Average Packet Latency under Uniform Random Traffic.

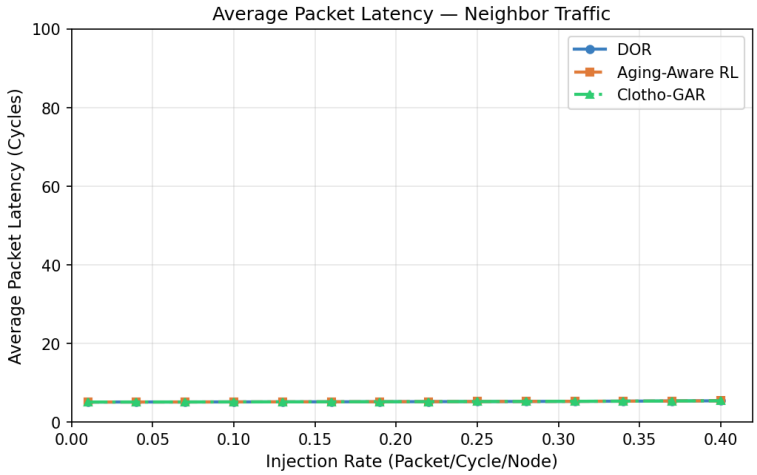


(b) Average Packet Latency under Shuffle Traffic.

Figure 4.5: Average Packet Latency vs. Injection Rate for RL-based Routing and DOR across Uniform Random and Shuffle traffic patterns.



(a) Average Packet Latency under Tornado Traffic.



(b) Average Packet Latency under Neighbor Traffic.

Figure 4.6: Average Packet Latency vs. Injection Rate for RL-based Routing and DOR across Tornado and Neighbor traffic patterns.

Chapter 5

Limitations and Future Work

5.1 Limitations

While the proposed RL-based routing framework demonstrates strong improvements in reliability and competitive performance, several limitations remain.

First, the evaluation is conducted using synthetic traffic patterns. Although these patterns provide controlled and diverse stress scenarios, they do not fully capture the complexity and temporal variability of real application workloads [48]. Second, the current formulation focuses primarily on reliability and latency. Other important system-level metrics, such as energy consumption and power efficiency [49], are not explicitly considered in the routing policy. Third, the study is limited to a 2D mesh NoC topology. Emerging architectures such as 2.5D and 3D NoCs [8] introduce additional challenges, including increased thermal coupling and vertical interconnect constraints, which are not captured in this work. Finally, the proposed approach is evaluated in a simulation environment. While the use of a cycle-accurate simulator [17] provides detailed insights, hardware-level validation is necessary to assess practical implementation overheads and feasibility.

5.2 Future Work

There are several promising directions for extending this work. A natural next step is to evaluate the proposed routing framework using real application workloads, such as PARSEC [50] and SPLASH-2 [51], to better understand its behavior under realistic traffic conditions.

The reinforcement learning formulation can be extended to incorporate additional objectives, such as energy efficiency, enabling a more comprehensive multi-objective optimization framework. In particular, replacing the fixed weighted-sum reward function with a dynamic or Pareto-based approach

[52] would allow the routing policy to adaptively balance trade-offs between reliability and performance. Future work can also explore the application of this approach to emerging NoC architectures, including 2.5D and 3D designs [8], where reliability and thermal challenges are more pronounced. Finally, implementing and validating the proposed RL-based routing policy in hardware would provide valuable insights into its practical overheads, scalability, and real-world applicability.

Together, these directions provide a pathway toward more adaptive, efficient, and reliable NoC routing strategies in future many-core systems.

Chapter 6

Conclusion

This thesis presented a reinforcement learning-based routing framework for improving the reliability of NoC interconnects under aging effects. The proposed approach integrates device-level aging models for EM and HCI into the routing decision process, enabling routers to dynamically adapt traffic distribution based on both performance and reliability considerations. Unlike prior aging-aware routing techniques that rely on fixed heuristics, the proposed method employs linear function approximation-based reinforcement learning to learn routing policies online during execution. This allows the routing strategy to adapt to changing traffic patterns and evolving network conditions without requiring explicit modeling of all possible scenarios.

Evaluation results demonstrate that the RL-based routing approach significantly improves system lifetime compared to deterministic routing, achieving up to $\sim 5\times$ improvement in EM lifetime and consistent gains in HCI lifetime across multiple traffic patterns. When compared to the state-of-the-art Clotho-GAR approach, the proposed method achieves comparable or superior reliability improvements in most traffic patterns, particularly under asymmetric and structured traffic conditions.

In terms of performance, the RL-based approach maintains latency comparable to Clotho and consistently improves over DOR by delaying network saturation. While Clotho exhibits slightly better latency under certain traffic patterns such as transpose and bit reverse, the RL-based method provides more consistent performance across a wider range of traffic conditions. These results highlight a key trade-off. Heuristic-based routing can be optimized for specific patterns, whereas learning-based routing offers greater robustness and adaptability.

Overall, this work demonstrates that reinforcement learning is an effective and flexible framework for aging-aware routing in NoCs, offering improved reliability while maintaining competitive performance.

Bibliography

[1] W.J. Dally and B. Towles. Route packets, not wires: on-chip interconnection networks. In *Proceedings of the 38th Design Automation Conference (IEEE Cat. No.01CH37232)*, pages 684–689, 2001.

[2] Arseniy Vitkovskiy, Vassos Soteriou, and Paul V. Gratz. Clotho: Proactive wearout deceleration in chip-multiprocessor interconnects. In *2015 33rd IEEE International Conference on Computer Design (ICCD)*, pages 117–124, 2015.

[3] J.R. Black. Electromigration—a brief survey and some recent results. *IEEE Transactions on Electron Devices*, 16(4):338–347, 1969.

[4] JEDEC Solid State Technology Association. Failure Mechanisms and Models for Semiconductor Devices. JEDEC Standard JESD122G, 2010. Available at: https://www.jedec.org.

[5] Joseph B. Bernstein. Chapter 3 - failure mechanisms. In *Reliability Prediction from Burn-In Data Fit to Reliability Models*, pages 31–48. Academic Press, Oxford, 2014.

[6] Hyungjun Kim, Arseniy Vitkovskiy, Paul V. Gratz, and Vassos Soteriou. Use it or lose it: Wear-out and lifetime in future chip multiprocessors. In *2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 136–147, 2013.

[7] Vijeta Rathore, Vivek Chaturvedi, Amit K. Singh, Thambipillai Srikanthan, and Muhammad Shafique. Life guard: A reinforcement learning-based task mapping strategy for performance-centric aging management. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2019.

[8] Lashmi Kondoth, Rajan Shankaran, Quan Z. Sheng, Endrowednes Kuantama, and Wei Ni. A survey of adaptive routing techniques in

two-dimensional network-on-chip architectures. *IEEE Communications Surveys & Tutorials*, 28:3195–3234, 2026.

[9] Rajendra M Patrikar Shubhangi D Chawade, Mahendra A Gaikwad. Review of xy routing algorithm for network-on-chip architecture. *International Journal of Computer Applications*, 43(21):20–23, April 2012.

[10] Muhammad Adnan, Muhammad Akmal Chaudary, Muhammad Moazzam Ali, Hafiz Ayaan Aatif, Muhammad Ibrahim Raza, and Hafiz Arslan Ramzan. Fault-tolerant adaptive routing in nocs: Machine learning approaches for resilient on-chip networks. In *2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*, pages 1–7, 2025.

[11] Jagadheesh Samala, Harshvardhan Takawale, Yash Chokhani, P. Veda Bhanu, and Soumya J. Fault-tolerant routing algorithm for mesh based noc using reinforcement learning. In *2020 24th International Symposium on VLSI Design and Test (VDAT)*, pages 1–6, 2020.

[12] Samala Jagadheesh, P. Veda Bhanu, J. Soumya, and Linga Reddy Cenkeramaddi. Reinforcement learning based fault-tolerant routing algorithm for mesh based noc and its fpga implementation. *IEEE Access*, 10:44724–44737, 2022.

[13] Zana Ghaderi, Ayed Alqahtani, and Nader Bagherzadeh. Aroma: Aging-aware deadlock-free adaptive routing algorithm and online monitoring in 3d nocs. *IEEE Transactions on Parallel and Distributed Systems*, 29(4):772–788, 2018.

[14] Kshitij Bhardwaj, Koushik Chakraborty, and Sanghamitra Roy. Towards graceful aging degradation in nocs through an adaptive routing algorithm. In *DAC Design Automation Conference 2012*, pages 382–391, 2012.

[15] Shaocong Wang, Xiaoyun Zhang, Dezun Dong, Cunlu Li, Zicong Wang, and Zongmao Zhang. Lare: A linear approximate reinforcement learning based adaptive routing for network-on-chips. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2023.

[16] J. Duato. A new theory of deadlock-free adaptive routing in wormhole networks. *IEEE Transactions on Parallel and Distributed Systems*, 4(12):1320–1331, 1993.

[17] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *SIGARCH Comput. Archit. News*, 39(2):1–7, August 2011.

[18] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. The gem5 simulator: Version 20.0+, 2020.

[19] Jingcao Hu and R. Marculescu. Dyad - smart routing for networks-on-chip. In *Proceedings. 41st Design Automation Conference, 2004.*, pages 260–263, 2004.

[20] Sheng Ma, Natalie Enright Jerger, and Zhiying Wang. Dbar: An efficient routing algorithm to support multiple concurrent applications in networks-on-chip. In *2011 38th Annual International Symposium on Computer Architecture (ISCA)*, pages 413–424, 2011.

[21] Paul Gratz, Boris Grot, and Stephen W. Keckler. Regional congestion awareness for load balance in networks-on-chip. In *2008 IEEE 14th*

International Symposium on High Performance Computer Architecture, pages 203–214, 2008.

[22] Sheng-Chun Kao, Chao-Han Huck Yang, Pin-Yu Chen, Xiaoli Ma, and Tushar Krishna. Reinforcement learning based interconnection routing for adaptive traffic optimization. *CoRR*, abs/1908.04484, 2019.

[23] Md Farhadur Reza and Tung Thanh Le. Reinforcement learning enabled routing for high-performance networks-on-chip. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2021.

[24] Shaocong Wang, Xiaoyun Zhang, Changhong Wang, Ke Wu, Cunlu Li, and Dezun Dong. Drlar: A deep reinforcement learning-based adaptive routing framework for network-on-chips. *Computer Networks*, 246:110419, 2024.

[25] Masoumeh Ebrahimi, Masoud Daneshtalab, Fahimeh Farahnakian, Juha Plosila, Pasi Liljeberg, Maurizio Palesi, and Hannu Tenhunen. Haraq: Congestion-aware learning model for highly adaptive routing algorithm in on-chip networks. In *2012 IEEE/ACM Sixth International Symposium on Networks-on-Chip*, pages 19–26, 2012.

[26] Hongzhi Zhao, Nader Bagherzadeh, and Jie Wu. A general fault-tolerant minimal routing for mesh architectures. *IEEE Transactions on Computers*, 66(7):1240–1246, 2017.

[27] Chaochao Feng, Zhonghai Lu, Axel Jantsch, Jinwen Li, and Minxuan Zhang. Fon: Fault-on-neighbor aware routing algorithm for networks-on-chip. In *23rd IEEE International SOC Conference*, pages 441–446, 2010.

[28] José Flich, Samuel Rodrigo, and José Duato. An efficient implementation of distributed routing algorithms for nocs. In *Second ACM/IEEE International Symposium on Networks-on-Chip (nocs 2008)*, pages 87–96, 2008.

[29] Anugrah Jain, Vijay Laxmi, Meenakshi Tripathi, Manoj Singh Gaur, and Rimpy Bishnoi. Track: An algorithm for fault-tolerant, dynamic and scalable 2d mesh network-on-chip routing reconfiguration. *Integration*, 72:92–110, 2020.

[30] Liang Wang, Xiaohang Wang, and Terrence Mak. Adaptive routing algorithms for lifetime reliability optimization in network-on-chip. *IEEE Transactions on Computers*, 65(9):2896–2902, 2016.

[31] Liang Wang, Xiaohang Wang, Ho-Fung Leung, and Terrence Mak. A non-minimal routing algorithm for aging mitigation in 2d-mesh nocs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 38(7):1373–1377, 2019.

[32] Dean Michael Ancajas, Koushik Chakraborty, Sanghamitra Roy, and Jason Allred. Tackling qos-induced aging in exascale systems through agile path selection. In *2014 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, pages 1–10, 2014.

[33] Zana Ghaderi, Ayed Alqahtani, and Nader Bagherzadeh. Online monitoring and adaptive routing for aging mitigation in nocs. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 67–72, 2017.

[34] Christopher Watkins. Learning from delayed rewards. 01 1989.

[35] Todd Hester, Matej Vecerík, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John P. Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Learning from demonstrations for real world reinforcement learning. *CoRR*, abs/1704.03732, 2017.

[36] Kelin J. Kuhn, Martin D. Giles, David Becher, Pramod Kolar, Avner Kornfeld, Roza Kotlyar, Sean T. Ma, Atul Maheshwari, and Sivakumar Mudanai. Process technology variation. *IEEE Transactions on Electron Devices*, 58(8):2197–2208, 2011.

[37] Smruti R. Sarangi, Brian Greskamp, Radu Teodorescu, Jun Nakano, Abhishek Tiwari, and Josep Torrellas. Varius: A model of process variation and resulting timing errors for microarchitects. *IEEE Transactions on Semiconductor Manufacturing*, 21(1):3–13, 2008.

[38] Mark White and Joseph B. Bernstein. Microelectronics reliability: Physics-of-failure based modeling and lifetime evaluation. Technical Report JPL Publication 08-5, Jet Propulsion Laboratory, California Institute of Technology, February 2008. Accessed: 2026-04-25.

[39] Fabrizio Mulas, David Atienza, Andrea Acquaviva, Salvatore Carta, Luca Benini, and Giovanni De Micheli. Thermal balancing policy for multiprocessor stream computing platforms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 28(12):1870–1882, 2009.

[40] Jason Howard, Saurabh Dighe, Sriram R. Vangal, Gregory Ruhl, Nitin Borkar, Shailendra Jain, Vasantha Erraguntla, Michael Konow, Michael Riepen, Matthias Gries, Guido Droege, Tor Lund-Larsen, Sebastian Steibl, Shekhar Borkar, Vivek K. De, and Rob Van Der Wijngaart. A 48-core ia-32 processor in 45 nm cmos using on-die message-passing and dvfs for performance and power scaling. *IEEE Journal of Solid-State Circuits*, 46(1):173–183, 2011.

[41] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

[42] Richard E. Bellman and Stuart E. Dreyfus. *Applied Dynamic Programming*. Princeton University Press, 1962.

[43] Syeda Tanjila Atik, M.M. Imran, Julkar N Mahi, Jenia A Jeba, Z I Chowdhury, and M S Kaiser. An adaptive routing algorithm for on-chip 2d mesh network with an efficient buffer allocation scheme. In *2018 International Conference on Computer, Communication, Chemical, Material and Electronic Engineering (IC4ME2)*, pages 1–4, 2018.

[44] Yoshiki Jojima and Masaru Fukushi. A fault-tolerant routing method for 2d-mesh network-on-chips based on components of a router. In *2016 IEEE 5th Global Conference on Consumer Electronics*, pages 1–2, 2016.

[45] Pavel Tvrdik. Cs 838: Topics in parallel computing, section 8: Virtual channels and deadlocks. Lecture notes, University of Wisconsin–Madison, Computer Sciences Department, 1999. Spring 1999.

[46] Dally and Seitz. Deadlock-free message routing in multiprocessor interconnection networks. *IEEE Transactions on Computers*, C-36(5):547–553, 1987.

[47] Wang Zhang, Ligang Hou, Jinhui Wang, Shuqin Geng, and Wuchen Wu. Comparison research between xy and odd-even routing algorithm of a 2-dimension 3x3 mesh topology network-on-chip. In *2009 WRI Global Congress on Intelligent Systems*, volume 3, pages 329–333, 2009.

[48] Mikhail A. Malikov and Aleksandr Y. Romanov. Traffic patterns in networks-on-chip: A survey. *IEEE Access*, 13:148803–148815, 2025.

[49] Ji Wu, Dezun Dong, and Li Wang. Noc power optimization using combined routing algorithms. In *2017 IEEE/ACIS 16th International Conference on Computer and Information Science (ICIS)*, pages 299–304, 2017.

[50] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. The parsec benchmark suite: characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, PACT '08, page 72–81, New York, NY, USA, 2008. Association for Computing Machinery.

[51] S.C. Woo, M. Ohara, E. Torrie, J.P. Singh, and A. Gupta. The splash-2 programs: characterization and methodological considerations. In *Proceedings 22nd Annual International Symposium on Computer Architecture*, pages 24–36, 1995.

[52] Ryan Bahlous-Boldi, Li Ding, Lee Spector, and Scott Niekum. Pareto-optimal learning from preferences with hidden context, 2025.

Appendix A

Mean Acceleration Factor

A.1 Mean Acceleration Factor (AF) Trends Across Injection Rates

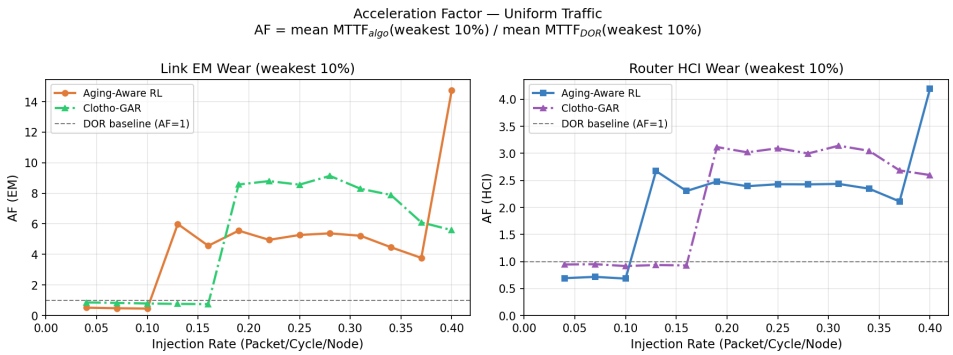


Figure A.1: Mean Acceleration Factor (AF) for uniform random traffic across different injection rates.

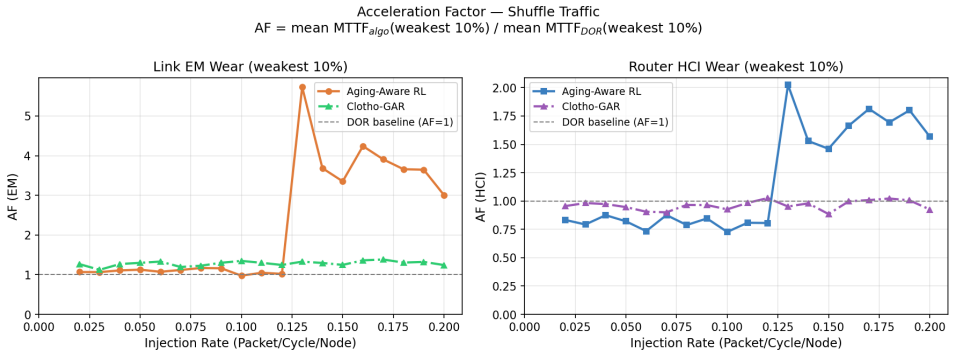


Figure A.2: Mean Acceleration Factor (AF) for shuffle traffic across different injection rates.

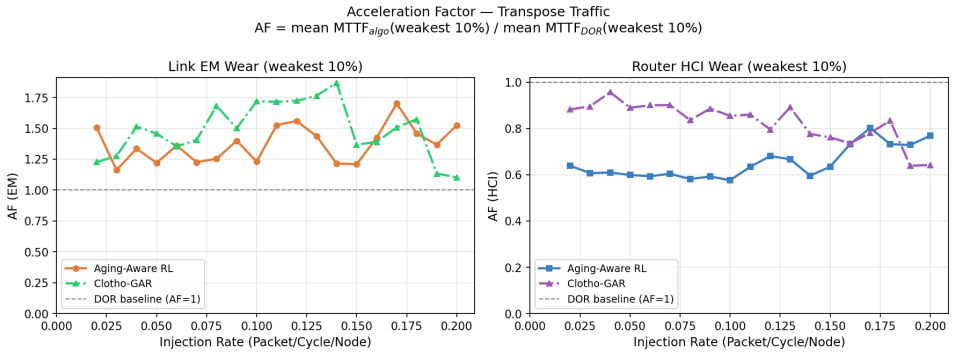


Figure A.3: Mean Acceleration Factor (AF) for transpose traffic across different injection rates.

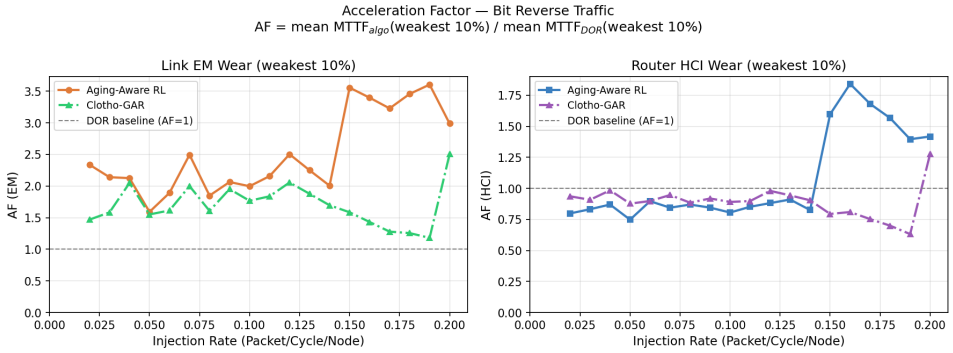


Figure A.4: Mean Acceleration Factor (AF) for bit reverse traffic across different injection rates.

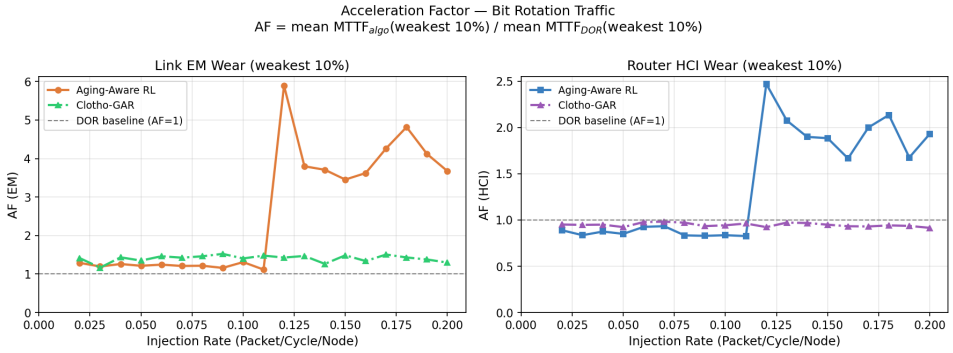


Figure A.5: Mean Acceleration Factor (AF) for bit rotation traffic across different injection rates.

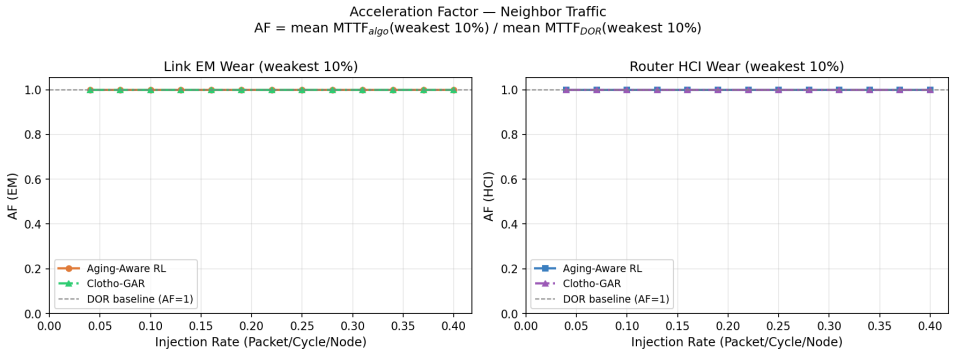


Figure A.6: Mean Acceleration Factor (AF) for neighbor traffic across different injection rates.

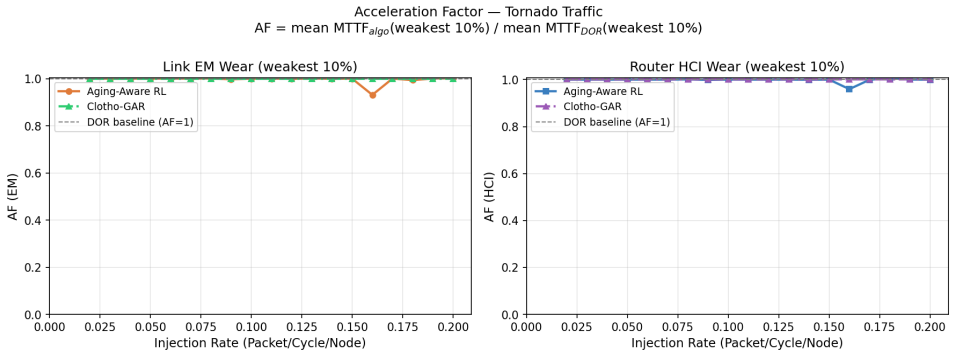


Figure A.7: Mean Acceleration Factor (AF) for tornado traffic across different injection rates.

Appendix B

Access to Research Work

B.1 Code

The code and relevant documentation for this thesis are available at the following GitHub repository: `https://github.com/IshraqF/CPEN-499-Thesis.git`. This repository includes all source code, scripts, and data used in the experiments and analyses presented in this thesis. The repository is organized into directories corresponding to the simulator, the thesis report, and the thesis logbook, ensuring that all materials are easily accessible for review and replication of results.